Quizs

Administrator and Application Documentation

By: **Arknox**

Email: info@arknox.in

Contact: +91-8787388949

Quizs combines a Laravel administrator panel and API with a Flutter quiz application. This guide explains setup, content management, account controls, notifications, settings, environment variables, and the sources behind app statistics.

Read the step-by-step workflows, field references, screenshots, and verification notes before changing a feature. Administrator saves, server configuration, and app releases affect different parts of the system.

Administrator Features

Manage categories, quizzes, bilingual questions, attempts, users, attendance, notifications, and default daily targets. Inspect analytics and reporting using their actual database definitions.

Application Features

Follow authentication, quiz answering, explanations, results, profile statistics, attendance, achievements, and inbox flows. Native push, sign-in, ads, and app updates require separate device/provider verification.

How to use this document

The complete PDF is continuous and includes a contents page and PDF bookmarks. The index.html reader adds sidebar navigation. Download the full manual or one chapter; both use the same official document format.

Screenshot provenance

Screenshots use this project’s local demo data. The cPanel setup map is illustrative. The faint logo, publisher header, purple headings, justified body text, and conclusion treatment follow the supplied official format reference.

Contents

Server & System Requirements 4

Deploy Quizs on cPanel, step by step 5

Local Terminal / CLI Setup (Quick Commands) 9

Understand the dashboard 10

Create and edit categories 14

Create and publish a quiz 17

Add questions and correct answers 20

Inspect quiz attempts and reports 23

Manage user accounts and custom targets 26

Send notifications 29

Settings: every customization and its storage 33

Read daily targets and history 36

Mark attendance 39

Configure and inspect offerwall completions 41

Search and filter administrator lists 43

Set the global daily target 46

Configure email and OTP delivery 47

Configure Firebase push credentials 49

Configure administrator login CAPTCHA 51

Apply database migrations 53

Customize settings with environment variables 55

Walk through the app screens 62

Understand every profile statistic and setting 70

Understand results and leaderboards 73

Read attendance and attendance rate 75

Read achievements and global ranking 77

Use the inbox and read tracking 79

Find every customization point 81

Troubleshoot common problems 84

Flutter Mobile Application Setup 86

Firebase & Google Sign-In Fix (SHA-1 Setup) 87

Configure banner and interstitial ads 88

Compiling APK & Play Store (AAB) 89

Software Delivery Agreement & Copyright Notice 90

Server & System Requirements

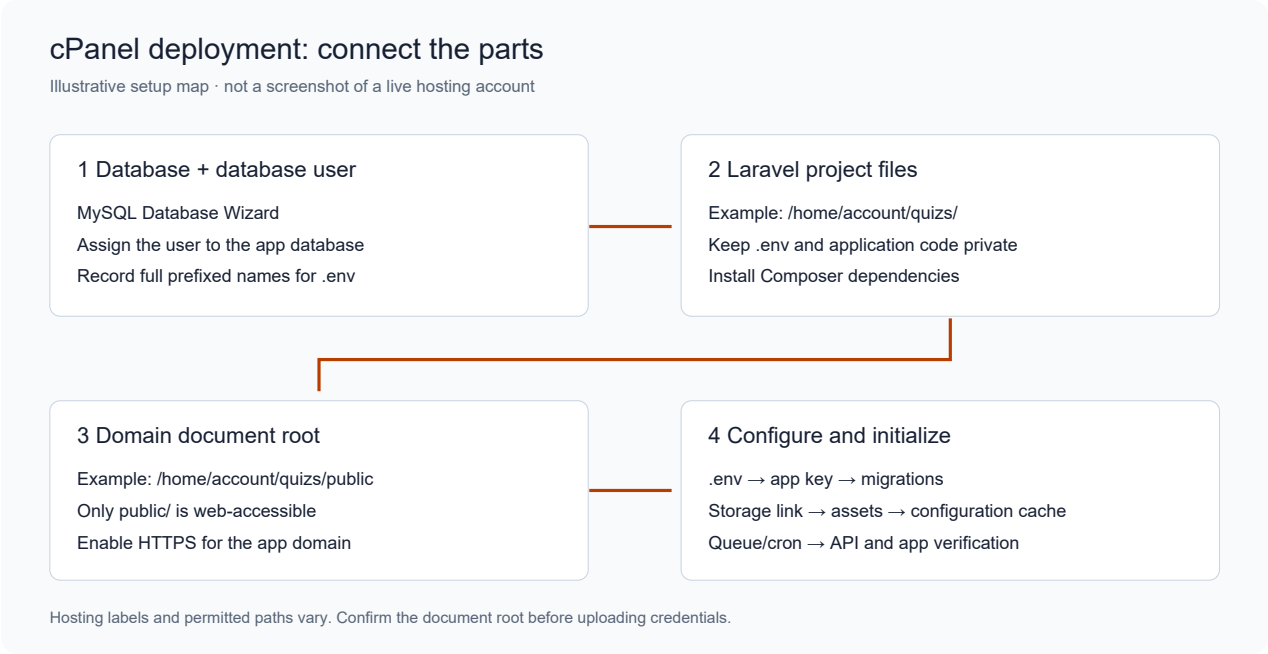
Before starting the installation, ensure your hosting server or local workstation satisfies the required technical stack.

Requirement	Minimum	Recommended	Notes
PHP Version	8.2.0	PHP 8.2 or 8.3	Required by Laravel 12 framework.
Database Engine	MySQL 8.0+ / MariaDB 10.4+	MySQL 8.0+	Must support `utf8mb4_unicode_ci`.
Web Server	Apache 2.4+ / Nginx 1.20+	Apache with mod_rewrite	Document root must point to `public/`.
PHP Extensions	`BCMath`, `Ctype`, `cURL`, `DOM`, `Fileinfo`, `JSON`, `Mbstring`, `OpenSSL`, `PCRE`, `PDO`, `PDO_MySQL`, `Tokenizer`, `XML`.		
Flutter SDK	Flutter with Dart 3.13.1 or compatible	Match pubspec.yaml and the lockfile	The app declares Dart SDK ^3.13.1. Verify flutter --version before installing dependencies.

Deploy Quizs on cPanel, step by step

Deploy the Laravel backend, connect its database, and expose only the public directory. The local diagram explains the setup; it is not a capture of your hosting account. cPanel labels and available tools can vary by host.

Screen / location: Hosting → Database / Files / Domain / Terminal



Illustrative setup map - not a live hosting screenshot

Step by step

1. Check that hosting supports the PHP version in composer.json, required extensions, a compatible database, Composer/SSH or a deployment artifact, and a domain document root that can point to the project's public directory.

2. Open MySQL Database Wizard (or the host's database tool). Create the app database and a database user, assign that user privileges on this app database, and record the full account-prefixed names.

3. Use File Manager to upload and extract Quiz Application Admin into a private application directory, for example /home/account/quizs. Keep .env, storage, vendor, and application source outside the public web root.

4. In Domains, set the application domain's document root to /home/account/quizs/public. If the host cannot expose only public/, resolve its supported Laravel deployment arrangement before putting credentials on the server. Enable HTTPS.

5. Create .env from .env.example, then set production environment, HTTPS APP_URL, and the exact database host/name/user/password supplied by the host. Configure SMTP, Firebase, and other integrations through their dedicated chapters.

6. From the Laravel project directory, install production dependencies, generate the application key only for a new installation, and apply migrations. Review seeders before using sample/default data; never run fresh/reset commands against a live database.

7. Create the storage link and deploy built frontend assets. Give storage and bootstrap/cache appropriate write access. Use configuration caching after the environment is complete.

8. Run the configured queue processing method for queued mail, then verify /admin/login, /api/categories, public artwork URLs, and an app quiz attempt against this HTTPS domain.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Database names / user	Full names assigned by the cPanel account, often account-prefixed.	Must match DB_DATABASE and DB_USERNAME exactly.
Document root	The Laravel project's public/ directory only.	Prevents the web server from exposing .env and private application files.
APP_ENV / APP_DEBUG	production / false.	Production behavior without debug error details.
APP_URL	The final HTTPS application domain.	Affects generated public links and storage/image URLs.
DB_HOST / PORT	Values provided by the host; do not assume every host uses 127.0.0.1.	Determines the database connection endpoint.
APP_KEY	Generated for a fresh installation; retained on existing deployments.	Used for encryption. Replacing an existing key can invalidate encrypted data and sessions.
Storage and assets	public/storage link plus compiled public/build files.	Needed for uploaded artwork and the styled admin interface.
Queue processing	Queue worker or the project's configured cron runner.	Required for queued mail; an SMTP test alone does not start the worker.

Current implementation limit

A hosting diagram is not proof of a live deployment. Do not expose the entire Laravel project under a public document root. Do not apply default sample seeders blindly to an existing database.

Data and behavior in detail

Environment template

```
APP_NAME="Quizs"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://your-domain.example
DB_CONNECTION=mysql
DB_HOST=HOST_PROVIDED_DATABASE_HOST
DB_PORT=3306
DB_DATABASE=ACCOUNT_PREFIX_DATABASE
DB_USERNAME=ACCOUNT_PREFIX_USER
DB_PASSWORD=YOUR_PRIVATE_DATABASE_PASSWORD
```

Replace every placeholder privately on the server. Do not include live passwords in screenshots or documentation.

Fresh installation commands

```
composer install --no-dev --optimize-autoloader
php artisan key:generate
php artisan migrate --force
php artisan storage:link
php artisan config:cache
```

Run from the project directory after configuring .env. key:generate is for a new installation, not a routine upgrade. Use reviewed seeders only when their records are actually intended.

Build admin assets

```
npm ci
npm run build
```

If hosting has no Node runtime, build from the matching release on your workstation and upload the resulting public/build directory. Do not leave the production site dependent on a local development server.

Upgrades and background jobs

Back up before release. Preserve existing .env and APP_KEY, deploy the intended code/dependencies/assets, and apply pending migrations. Use the project's configured worker/cron method with the host's supported scheduler and private credentials. With QUEUE_CONNECTION=database, use a supervised php artisan queue:work worker, or the protected GET /api/cron/run-queue endpoint with the private CRON_SECRET configured in .env when persistent workers are unavailable. Keep the secret out of public links and screenshots. Check logs after the first job.

Verification after saving

Check login, API content, artwork, quiz submission, queue-dependent mail, and real-device push. Update ApiClient.baseUrl and build the app for the final backend domain.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

composer.json

.env.example

public/index.php

routes/web.php

app/Http/Controllers/CronController.php

Local Terminal / CLI Setup (Quick Commands)

If you are running the backend locally on XAMPP, Laragon, or an Ubuntu VPS via SSH, use these commands:

```
# 1. Navigate to backend directory:
cd "Quiz Application Admin"

# 2. Install PHP packages:
composer install

# 3. Create .env configuration:
cp .env.example .env
php artisan key:generate

# 4. Migrate database and seed default admin + sample quizzes:
php artisan migrate --seed

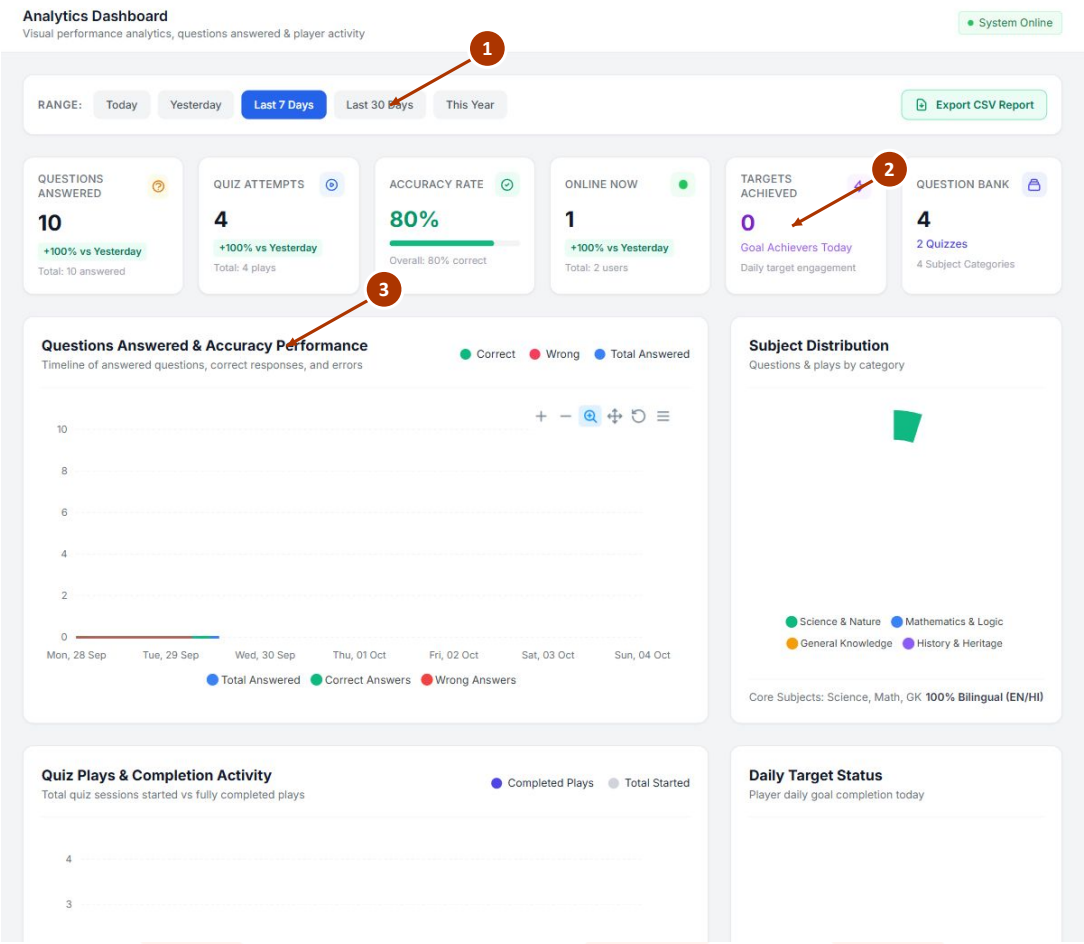
# 5. Link public storage for category/quiz images:
php artisan storage:link

# 6. Compile Tailwind CSS assets & start server:
npm install && npm run build
php artisan serve
```

Understand the dashboard

Learn how each number and chart is calculated, which records affect it, where to change the underlying data, and which labels are static interface text.

Screen / location: Admin → Dashboard



Actual source preview - local demo data - live values may differ

1. Date range and CSV export

2. Today's statistics

3. Questions and answers chart

Step by step

1. Open `/admin/login` on your server and sign in with an active administrator account.

2. Choose Today, Yesterday, Last 7 Days, Last 30 Days, or This Year. This changes chart time buckets; the main cards retain their today/yesterday and lifetime context.

3. Compare Total Answered, Correct, and Wrong in Questions Answered. Use the legend to toggle series and Zoom / Reset Zoom to change the report view.

4. Download the selected analytics using Export CSV Report. Open Recent Quiz Attempts → View all to inspect individual sessions.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Questions Answered	Sum of attempted_questions for attempts created today, with a lifetime total below. This is different from the number of questions in the bank.	Recorded answers and submissions affect tracking. This card is not an editable counter.
Quiz Attempts	Number of quiz_attempts started today. Starting and completing a quiz are separate events.	Starting a quiz in the app creates a play. Adding a question does not create one.
Accuracy Rate	Correct answers ÷ attempted questions × 100. When today has no answers, the card falls back to overall accuracy.	Recorded app answers are the source. Change future answer keys in Questions Bank.
Online Now	Users whose last_active_at falls within the configured timeout; ONLINE_TIMEOUT_SECONDS defaults to 120.	App heartbeats update activity. Change the timeout in config/quiz.php or the server environment, not an admin form.
Targets Achieved	Today's user_daily_progress records with completed status.	Daily target settings and completed quizzes affect this value. It is not a manually entered counter.
Question Bank	Counts of database questions, quizzes, and categories; inactive records may also be included.	Manage content through Categories, Quizzes, and Questions Bank.
Growth percentage	Today-versus-yesterday change. A zero previous value with a nonzero current value displays 100%. Growth beside Online refers to new users.	Read the label using its actual definition; new-user growth is different from live online activity.
System Online	A static layout badge, not an uptime probe or server health monitor.	The badge does not verify API availability, SMTP, or Firebase delivery.

Data and behavior in detail

Where the four charts get their data

Questions Answered and Quiz Plays group quiz_attempts.created_at into time buckets. Completed plays are completed attempts within those buckets. Subject Distribution counts questions across all categories, independent of the selected range. Daily Target Status always shows today, even when the timeline range changes.

Recent Attempts and New Players

Recent attempts are ordered by completed_at. New Players uses the latest user records and may include administrators. Points is an attempt score; accuracy is its correct-to-total question ratio.

Changing chart text

Edit category names and colors in Categories. Headings and captions such as “Core Subjects” and “100% Bilingual” are static strings in resources/views/admin/dashboard.blade.php, not live audit results. Changing those strings does not change app labels.

Read analytics one step at a time

1. Open Dashboard and choose Today, Yesterday, 7 Days, 30 Days, or This Year in the range bar. Wait for the charts to update before comparing results.

2. Hover over a timeline point to read its date and values. Compare adjacent dates to identify changes in activity; a zero can mean no recorded attempts in that period.

3. Use the timeline toolbar to zoom in, zoom out, pan, or reset the view. Reset restores the chart view; it does not reset application data. Click a legend item to hide or show that series.

4. Read the category chart as question distribution across categories. It is not a count of users or completed quizzes, and changing the date range does not restrict this distribution.

5. Click Export CSV Report after selecting the required range. Open the downloaded CSV to inspect the underlying report separately from a chart image downloaded through the chart toolbar.

6. To check a surprising value, open Quiz Attempts, filter the same dates, and inspect individual attempts. Today target cards and lifetime totals have their own scope; do not assume every card follows the chart range.

How to change what analytics shows

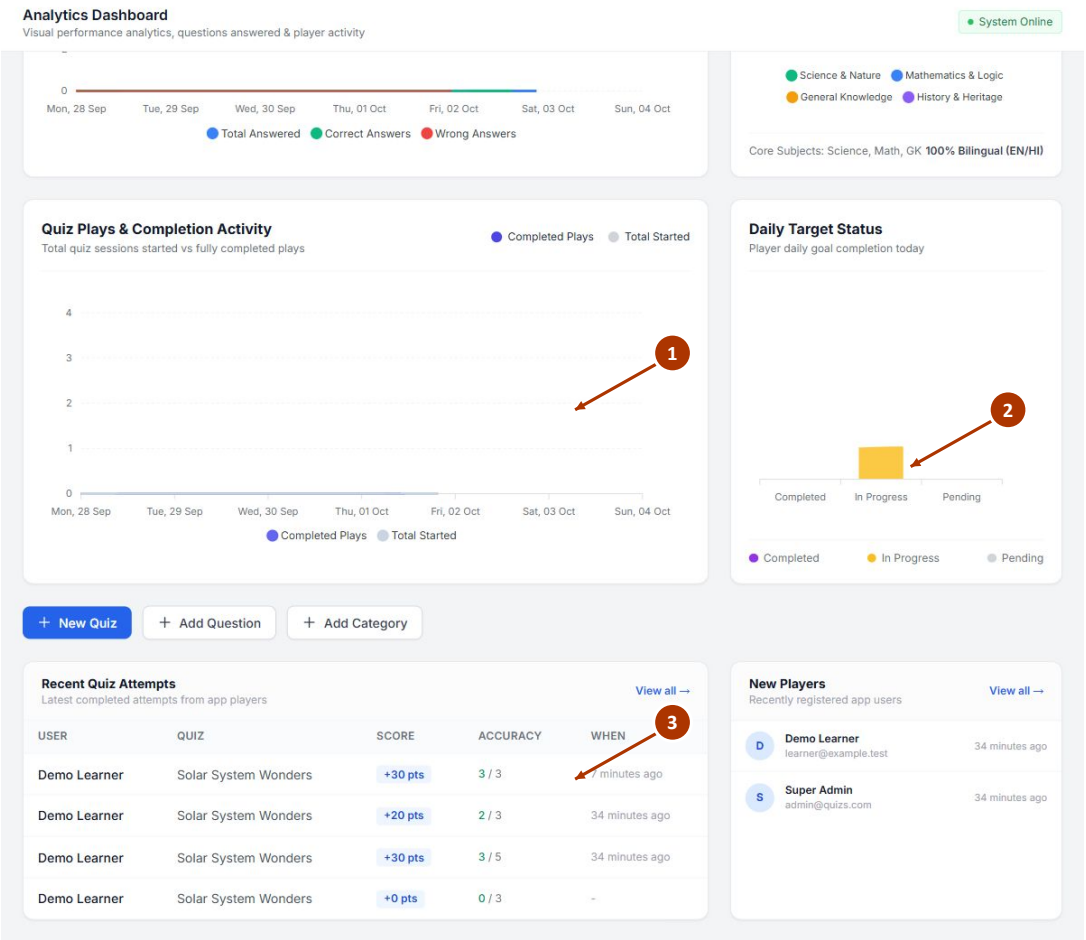
Add or edit content through Categories, Quizzes, and Questions. Learners generate attempt activity by answering and submitting quizzes. Dashboard charts read those stored records; there is no form for manually typing chart values. Refresh the dashboard after the corresponding activity. System Online and bilingual promotional labels are not live health or translation-completeness tests.

Verification after saving

Complete a quiz in the app, reload Dashboard, and compare attempt counts, answered questions, and correct/wrong tracking. Changing the reporting range does not modify user data.

Completion, targets, and recent activity

This lower dashboard capture shows quiz completion, today’s targets, recent attempts, and new players. Read each chart using its own time scope.



Actual source preview - local demo data - live values may differ

1. Started versus completed quizzes

2. Today's target status

3. Recent attempts and new users

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/DashboardController.php

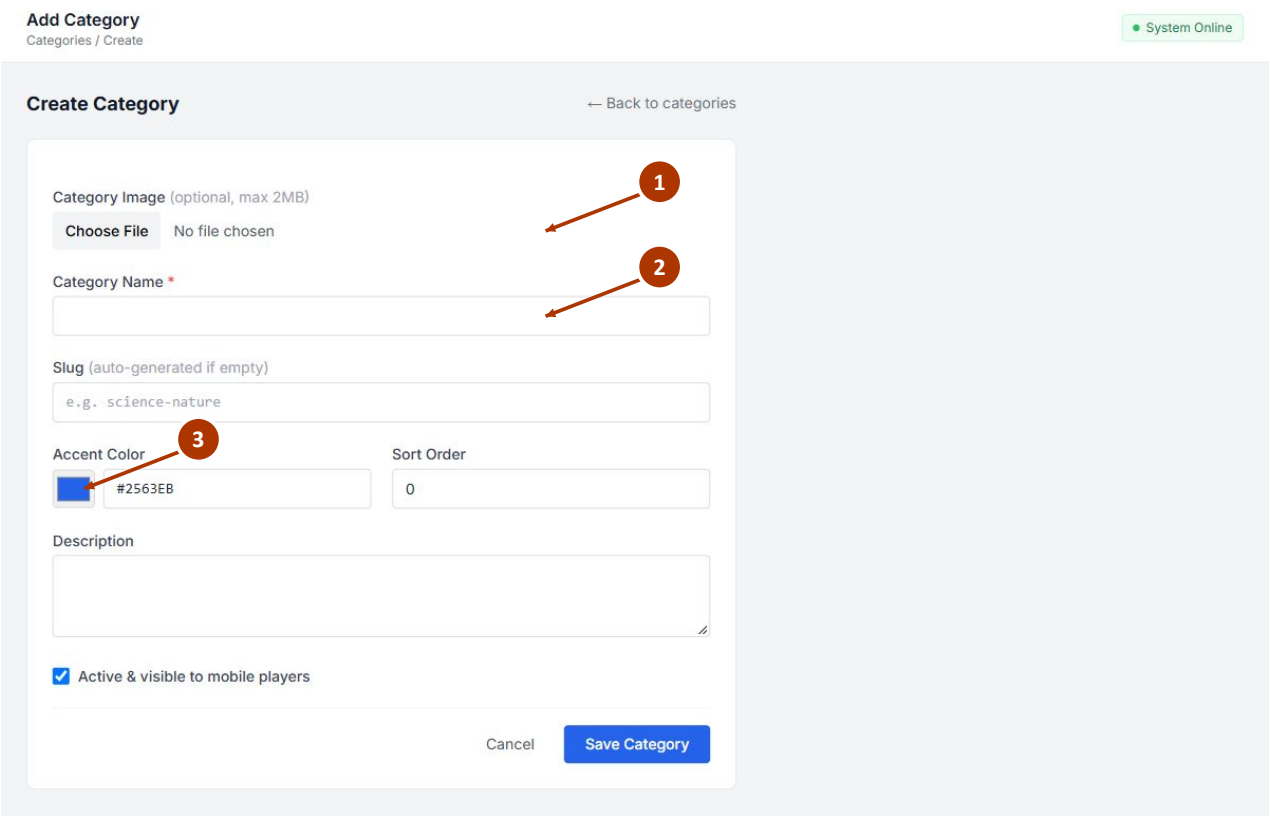
resources/views/admin/dashboard.blade.php

config/quiz.php

Create and edit categories

Categories organize subjects in the app. Names, artwork, colors, order, and visibility are fetched from the server when the category list loads.

Screen / location: Admin → Categories → Add Category



Actual source preview - local demo data - live values may differ

1. Category artwork

2. Name and identity

3. Color, order, and visibility

Step by step

1. Open Categories → Add Category.

2. Enter Category Name. Leave Slug blank to generate it from the name, or enter a unique custom slug.

3. Optionally upload an image up to 2 MB. Choose an Accent Color with the picker or hexadecimal value and enter Sort Order.

4. Enter an optional Description, select Active & visible to mobile players, and click Save Category.

5. Use Edit in the list to change saved fields, replace artwork, or remove the existing image.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Category Name	Required; maximum 255 characters.	Changes the app's category title and subject grouping.
Slug	Optional and unique; generated from the name when blank.	Changes server metadata, not the numeric ID used by API links.
Category Image	Optional image; maximum 2048 KB.	Provides image_url for app artwork. The public storage URL must be accessible.
Accent Color	Required color string; select a hexadecimal color in the picker.	Changes category color data. Each app component decides how to render that color.
Sort Order	Integer; smaller values appear first.	Changes the order of subjects returned by the API.
Description	Optional introductory text.	Changes API description data; not every app view displays it.
Active	Visibility switch.	Inactive categories disappear from the public category list without deleting their records.

Current implementation limit
Do not use Delete to hide content. Deleting a category can cascade to related quizzes and questions. Back up first; use Active for temporary visibility changes.

Data and behavior in detail

Add a category without missing a field

1. Open Categories and choose the add action. Enter the category name, optional unique slug, required color, and display order. Select an image only if you want category artwork.

2. Choose the active state deliberately: an inactive category should not be used as a published learner entry point.

3. Save the form. If validation appears, correct the indicated field and save again. Return to the list and check the name, active state, and order.

4. Open the app category list and reload its data to verify the result. Uploads are stored under public/uploads/categories; changing a documentation screenshot will not change app artwork.

Delete confirmation

Click Delete on the correct category row. The browser asks **Delete this category?** Choose **Cancel** to retain it or **OK** to submit deletion. Category deletion also removes dependent quizzes and questions. After successful deletion, the list refreshes; verify the row is gone before continuing.

Verification after saving

Reopen categories in the app and confirm the name and order. If artwork fails, check the storage link, APP_URL, and image URL.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/CategoryController.php

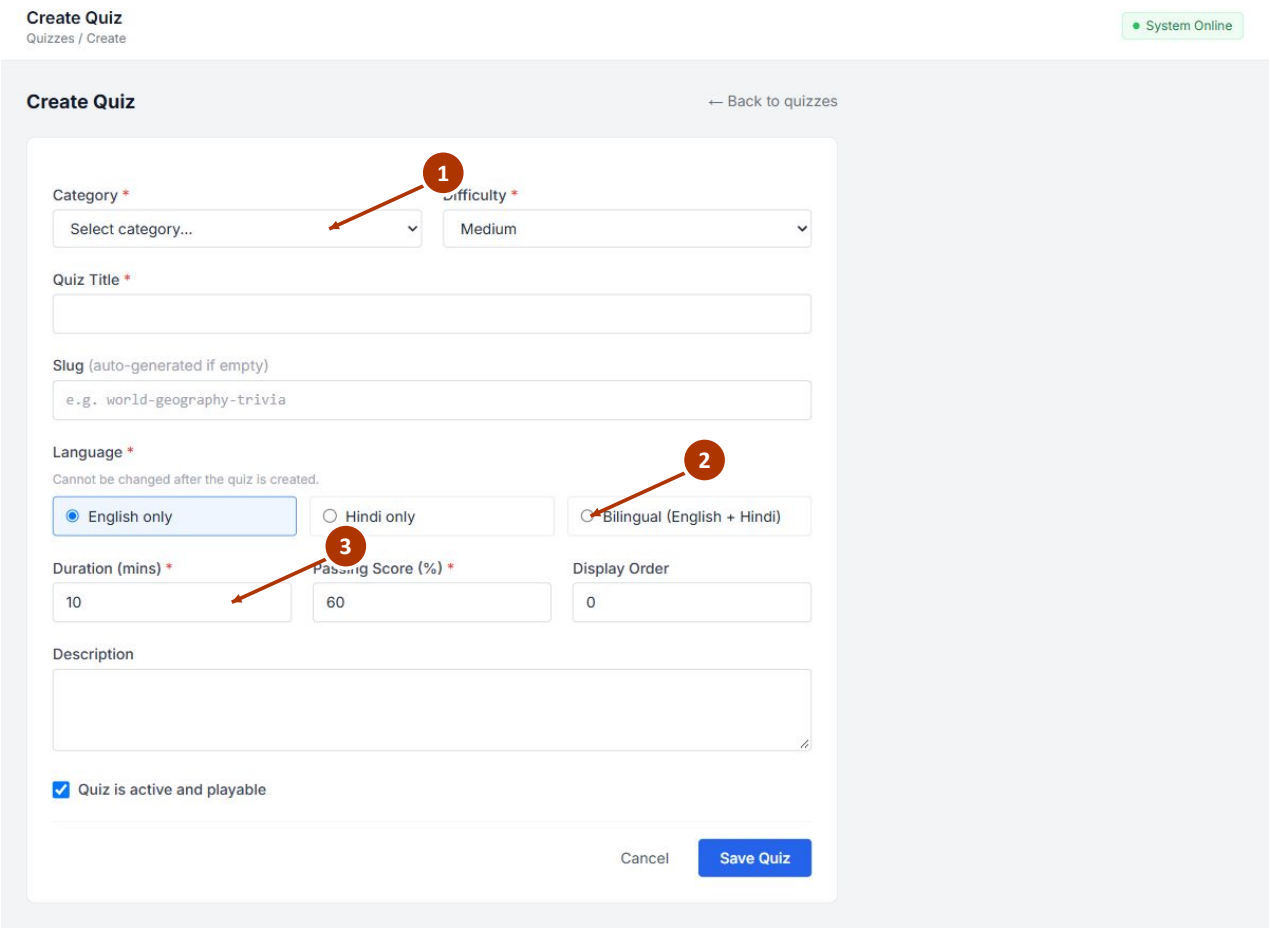
app/Http/Controllers/Api/QuizApiController.php

database/migrations/2026_09_16_104127_create_quizzes_table.php

Create and publish a quiz

Create a category first, then a quiz, then its questions. Language controls content selection; duration is saved metadata and is not currently an exam deadline in the Flutter question flow.

Screen / location: Admin → Quizzes → New Quiz



Actual source preview - local demo data - live values may differ

1. Category and difficulty

2. Quiz language

3. Duration, pass score, and active state

Step by step

1. Open New Quiz, select Category, and enter Quiz Title.

2. Choose Easy, Medium, or Hard. Select English only, Hindi only, or Bilingual at creation.

3. Enter Duration of at least 1 minute and Passing Score from 1-100. Set Description and Display Order.

4. Save the quiz as inactive while preparing questions and checking answer keys in Questions Bank.

5. Open Edit Quiz, enable Quiz is active and playable, and Save Quiz. Check that it appears under the correct category and loads questions in the app.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Category	Required existing category.	Determines the subject containing this quiz.
Title / Slug	Title: maximum 255; slug: optional and unique.	Changes the name and server metadata.
Difficulty	easy / medium / hard.	Changes difficulty metadata; it does not automatically rewrite questions.
Language	en / hi / bilingual on creation; bilingual is stored as null.	Single-language quizzes are filtered by language; bilingual quizzes use translations.
Duration	Integer, minimum 1; stored as duration_minutes.	The current Flutter flow does not use it as an exam deadline. Next Trial has a separate fixed 5-second countdown.
Passing Score	Percentage from 1-100.	The server compares correct answers ÷ total questions against this threshold. Passing increments streak; performance messages use separate thresholds.
Active / Display Order	Playability and ordering.	Availability and order change on the next content fetch.

Current implementation limit
Language cannot be changed through Edit Quiz: update validation excludes it. Create another quiz for a different language structure. Edit existing questions to change translations.

Data and behavior in detail

Played and completion rate

played_count is the lifetime number of unique users who completed this quiz. A repeat completion by the same person does not add another unique player. completion_rate reflects the current user's answered progress today, not the pass rate of all users.

Exactly what to choose in the form

1. Open the add quiz form. The Category dropdown starts with Select category... and contains saved category names. If your category is missing, create it in Categories first, then reload this form.

2. Choose Easy, Medium, or Hard from Difficulty. These are metadata labels; they do not automatically change questions or enforce a different scoring rule.

3. Choose the creation language: English, Hindi, or bilingual. Bilingual creation supports paired English and Hindi question content. The edit form does not provide the same language selector, so decide before adding questions.

4. Enter title, duration, passing percentage, and other required fields. Save the quiz, then open Questions and select this quiz to add its four-choice questions.

5. Activate the intended content and test it as a learner. A saved quiz without questions is not a finished quiz.

Filter, edit, and delete

On the quiz list, All Categories shows the complete list. Choose a saved category name to narrow the list. Edit opens the existing quiz; save and reload its details to verify changes. Delete asks **Delete this quiz and all its questions?** Choose **Cancel** to keep the quiz or **OK** to delete it and its questions. Do not use deletion merely to hide a quiz; change its active state instead.

Verification after saving

Open the app in the intended language. Confirm category, difficulty metadata, question order, and playable content. Populate an empty quiz before launch.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/QuizController.php

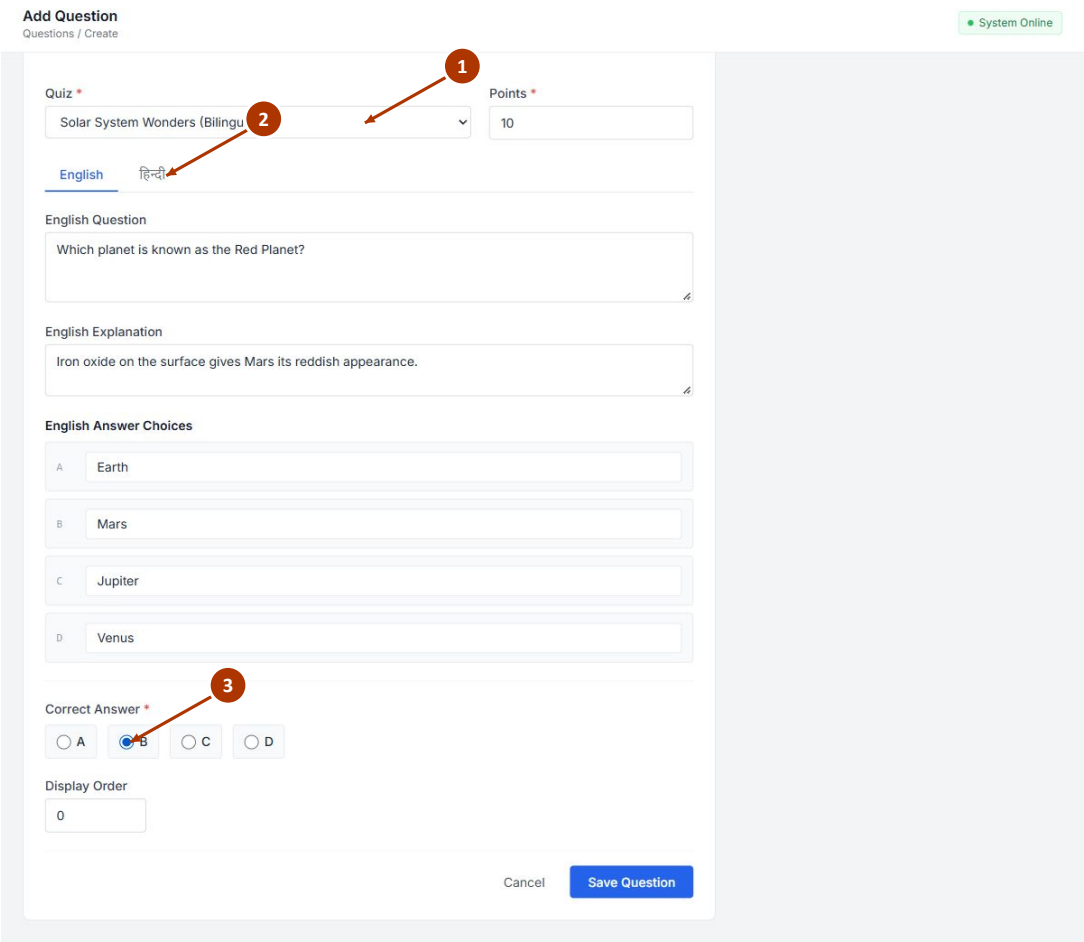
app/Services/QuizStatsService.php

lib/services/quiz_api_service.dart

Add questions and correct answers

This version supports text multiple-choice questions with four answer positions A-D and one correct position. English and Hindi translations in a bilingual quiz share the same correct position.

Screen / location: Admin → Questions Bank → Add Question



Actual source preview - local demo data - live values may differ

1. Quiz and points

2. English / Hindi translations

3. Correct option and Save

Step by step

1. Open Questions Bank → Add Question and choose the quiz receiving the question.

2. Enter Points, for example 10. For a single-language quiz, enter Question Text and an optional Explanation.

3. Fill all four choices A, B, C, and D. Select the Correct radio for the right answer; only one choice can be correct.

4. For a bilingual quiz, fill the English tab first, then the Hindi tab with translations in the same A-D positions.

5. Select the shared Correct Answer, enter Display Order, and click Save Question.

6. Use Edit to verify the saved content, then test the same quiz in both app languages.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Quiz	Required existing quiz. The form adapts to its language structure.	Adds the question to that quiz's question set.
Points	Required integer, minimum 1.	Changes points awarded for a correct answer. Progression XP and daily-target rewards are separate.
Question Text	The single-language question text.	Changes the main question shown in the app.
Explanation	Optional reasoning or learning text.	Returned by the question API and displayed by the app's answer/explanation flow.
Answer Choices	Four positions in the admin form.	Changes answer labels. There is no separate admin selector for true/false, multi-select, image, or audio question types.
Correct Answer	One A/B/C/D selection; backend position 0/1/2/3.	Sets the corresponding is_correct flag used for answer evaluation.
Display Order	Integer; default 0.	Changes the server-defined question order.

Current implementation limit
Editing question wording and changing its answer key have different effects. Consider active attempts and historical reports before changing a published key. Deleting a question is a data operation; refresh does not restore deleted content.

Data and behavior in detail

Complete example

Question: "Which planet is known as the Red Planet?" → A Earth, B Mars, C Jupiter, D Venus → Correct B → Points 10. In the Hindi translation, keep the same option order and the equivalent of Mars at B. The correct position remains B in both languages.

Missing Hindi translation

Complete English content first and add Hindi later. Missing requested translations use the configured fallback language, English by default. The server does not automatically translate missing content.

Previously submitted answers

Edited text and choices appear on the next API fetch. Existing submitted attempts are not automatically rescored. Option positions are synchronized on edits to preserve option IDs where possible.

Dropdowns and answer selection

1. On the question list, choose All Quizzes or a specific quiz title. Translation status offers Any Translation Status, Hindi Missing, and English Missing. Use a missing-language filter to find records requiring translation; it does not generate translations.

2. Open the add question form. Quiz starts with Select quiz... and lists saved quiz titles. Bilingual quizzes carry a (Bilingual) suffix. Select the quiz before entering question content because the visible language fields depend on it.

3. Enter the question and all four choices A, B, C, and D. Select exactly one correct-answer radio button. Choosing C means the third choice is correct in both language versions; it does not mean the English and Hindi text must be identical.

4. For a bilingual quiz, enter English question and choices, then add matching Hindi content. Keep answer positions aligned. Hindi can be completed later; missing content uses the existing fallback behavior.

5. Add an explanation, points, and display order as needed. Save, find the new row, and test the answer plus explanation in the app.

Worked example

Question: **What is 2 + 2?** Choices: A = 3, B = 4, C = 5, D = 6. Select **B** as Correct Answer and enter an explanation such as *Adding two and two gives four.* In Hindi, retain the same A-D positions. This is one single-answer text question; the form has no multi-select, audio, or image-question type dropdown.

Delete confirmation

Click the row's Delete action. The popup says **Delete this question?** Click **Cancel** to leave it unchanged or **OK** to submit deletion. Verify the refreshed list. Editing a correct answer changes future question behavior; it does not automatically rescore historical attempts.

Verification after saving

Try a correct answer and an incorrect answer and compare scores. Position B must mean the same answer in both languages. The screenshot is a locally filled example, not production content.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/QuestionController.php

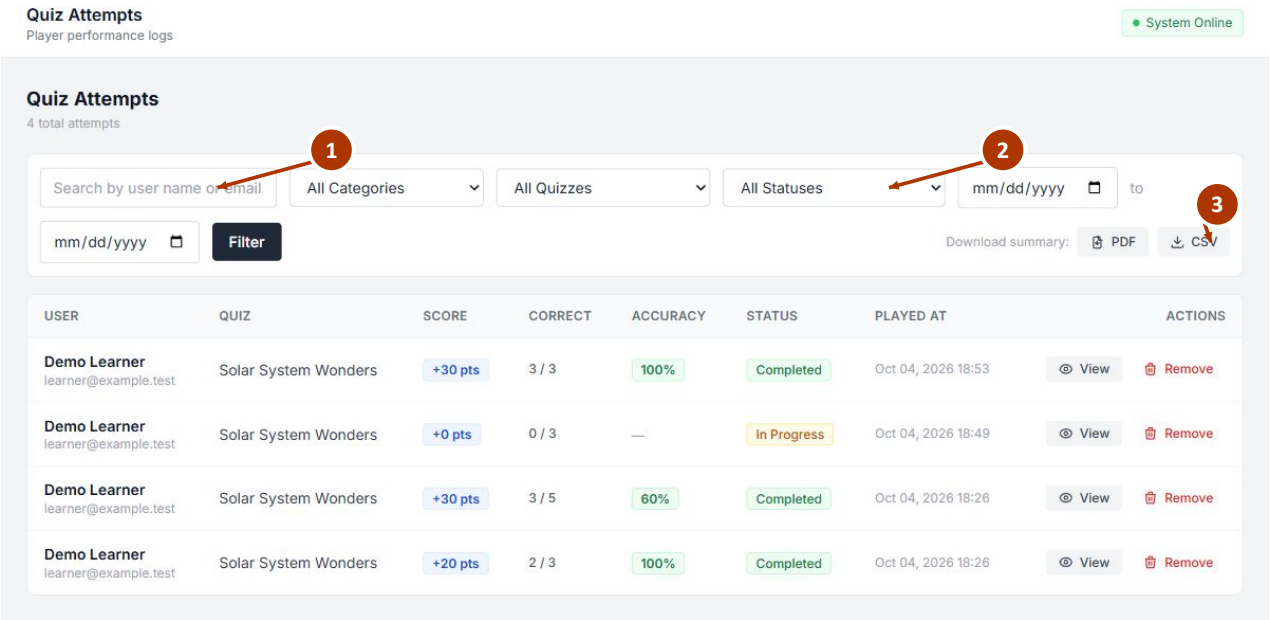
resources/views/admin/questions/create.blade.php

lib/models/question_model.dart

Inspect quiz attempts and reports

Attempts are app quiz sessions. Scores are calculated through answers and submission; this screen is not a form for manually creating results.

Screen / location: Admin → Quiz Attempts



Actual source preview - local demo data - live values may differ

1. Search and filters

2. Attempt status

3. View and export

Step by step

1. Filter Quiz Attempts by user, category, quiz, status, and date as needed.

2. Open View and read started/completed timestamps, elapsed time, answered count, and correct/wrong counts together.

3. Expand Toggle Questions to compare each selected answer with the correct answer.

4. Download individual PDF/CSV summaries or filtered list summaries. Check headings and filters to confirm the export scope.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Status	in_progress or completed.	Starting creates an attempt; submitting/completing marks it completed.
Score	Question points awarded for recorded correct answers.	Different from XP, total user points, and ranking position.
Answered / Correct / Wrong	attempted_questions, correct_answers, and wrong_answers tracking.	Incomplete attempts can affect some analytics totals.
Time	started_at, completed_at, and duration tracking.	Drives attempt timelines and ordering.
PDF / CSV	Individual and summary reports.	Downloading does not change app data.
Delete	Removes the attempt record.	Do not assume a score correction or target rollback; separate progress/XP records are not guaranteed to reverse automatically.

Current implementation limit
Changing a downloaded report does not change server or app results. Correct records through a real attempt or an appropriate backend correction.

Data and behavior in detail

Find and inspect a learner result

1. Enter a learner name or email in the search field. Choose All Categories or one saved category, and All Quizzes or one saved quiz.

2. Status offers All Statuses, Completed, and In Progress / Abandoned. Select Completed when checking final results. An unfinished attempt is not a completed result.

3. Set From and To dates for the period you need, then submit the filters. Clear irrelevant filters if an expected attempt is missing.

4. Open the matching attempt's detail action. Check learner, quiz, completion state, correct answers, score, and timing together. Inspect the answer breakdown before treating a score as an error.

5. Use the summary export action for the attempt report. This report describes learner activity; the documentation Chapter PDF button downloads instructions instead.

Delete an attempt log

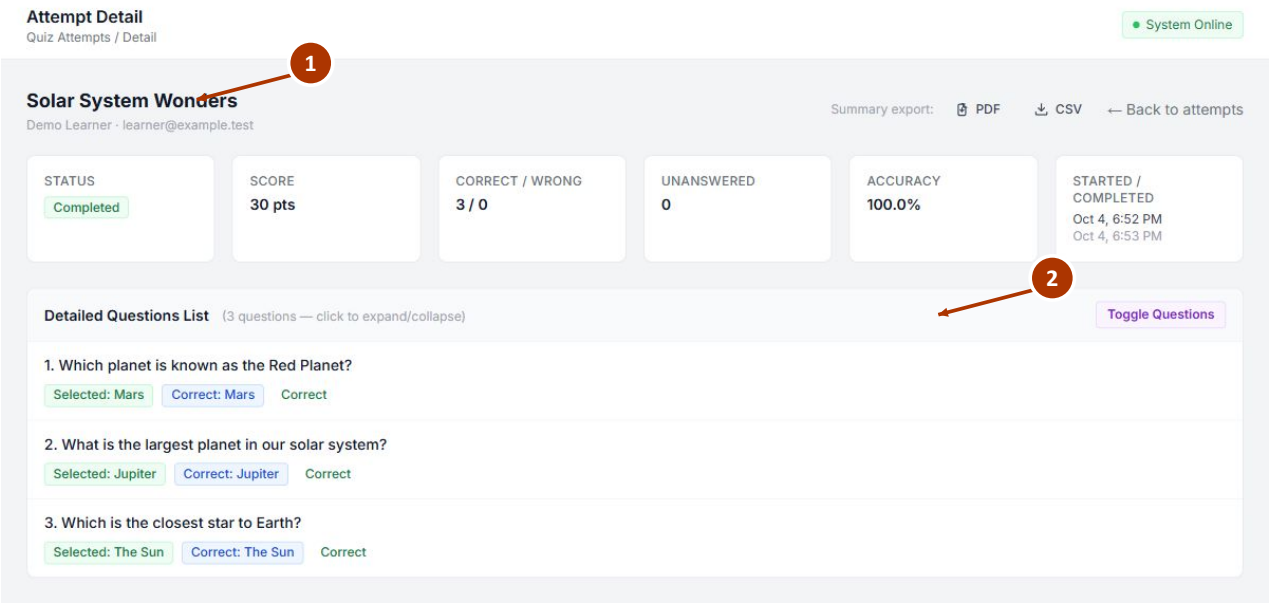
The Delete action asks **Delete this attempt log?** Select **Cancel** to retain it or **OK** to submit deletion. This is a record-management action, not a retry or score-edit button. Verify the row disappears and recheck any statistics that depend on attempt records.

Verification after saving

Compare the same demo user, quiz, answers, and score against the app result.

Answer breakdown for one attempt

Open View and expand Toggle Questions to compare selected and correct answers. PDF/CSV summarizes this attempt.



Actual source preview - local demo data - live values may differ

1. Quiz and summary exports

2. Expanded answer details

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

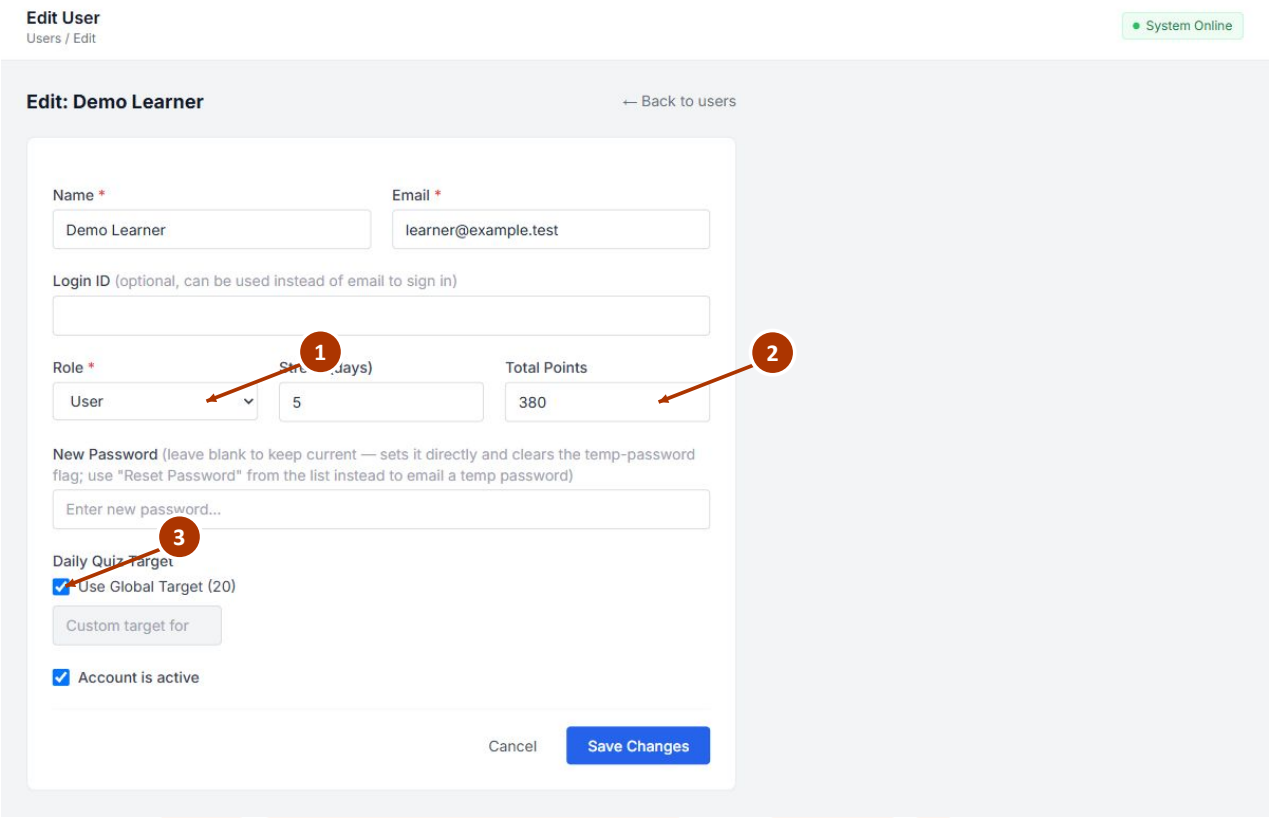
app/Http/Controllers/Admin/QuizAttemptController.php

app/Http/Controllers/Api/QuizApiController.php

Manage user accounts and custom targets

Edit identity, role, account activity, points, streak, and individual targets. Decide whether a change should affect everyone or one account.

Screen / location: Admin → Users → Edit



Actual source preview - local demo data - live values may differ

1. Identity and role

2. Points and streak

3. Global or custom target

Step by step

1. Search Users by name, email, or Login ID. Use Add User to create an administrator-managed account.

2. Enter name, unique email, optional unique Login ID, and role. The system generates a temporary password and attempts to send account email.

3. Edit profile fields. Leave Password blank to retain the existing password.

4. For an individual target, clear Use Global Target and enter 1-1000. Re-enable it to return to the global default.

5. Save Changes, refresh the app profile, and inspect Activity for sessions, last activity, and quiz tracking.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Name / Email / Login ID	Name/email required; email and login_id unique.	Updates identity on the next profile fetch; email and Login ID also affect sign-in.
Role	user or admin.	Admin grants management-panel access; it is not a cosmetic label.
Streak / Total Points	Nonnegative integers in users.streak / users.score.	Does not recalculate separate progression or report records. Despite the “Streak (days)” label, current logic increments for each passed quiz without checking consecutive calendar days.
New Password	Optional; minimum 6 characters.	Directly replaces the password and clears the temporary-password flag. Different from the list’s Reset Password action.
Reset Password	Creates and emails a new temporary password.	The user must complete Force Change Password at the next login.
Account is active	Activity switch.	Affects authentication and inclusion in eligible audience/ranking scopes.
Custom Daily Target	1-1000, or the global fallback.	Changes effective targets for newly created daily rows; existing daily history stays fixed.

Data and behavior in detail

Activity and temporary accounts

Activity records describe sessions and usage; they are not editable profile statistics. A must_change_password account must finish the password-change flow before unrestricted protected app actions. Account emails depend on mail transport and queue processing.

Role and presence dropdowns

The list offers **All Roles / Admin / User** and **Online or Offline / Online / Offline**. Combine these with search to locate an account. Online is estimated from last activity and timeout; it is not proof that the learner is looking at a particular screen. The add/edit role dropdown offers **User** and **Admin**. Choose User for a learner; Admin grants administrator access.

Create, edit, and verify an account

1. Choose the add action and complete the required identity, email, password, role, and active fields. Save and locate the new account in the list.

2. To change an existing account, open Edit, update only the intended fields, and save. Leave an optional password blank when retaining it. Check a custom daily target against the global fallback described in this chapter.

3. Open the activity action to inspect that learner’s attempts. Confirm the identity before changing a role or account state.

Reset password and delete popups

Reset Password asks **Reset password for [name]? A new temporary password will be emailed to them.** Select **Cancel** to stop or **OK** to proceed. Delivery requires working email settings, and the learner must change the temporary password. Delete asks **Delete user [name]?** Select **Cancel** to retain the account or **OK** to submit deletion. Read the displayed name before confirming either action.

Verification after saving

Refresh the intended user’s profile and verify identity, access, and target. Do not treat a total-score edit as an XP or level edit.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/UserController.php

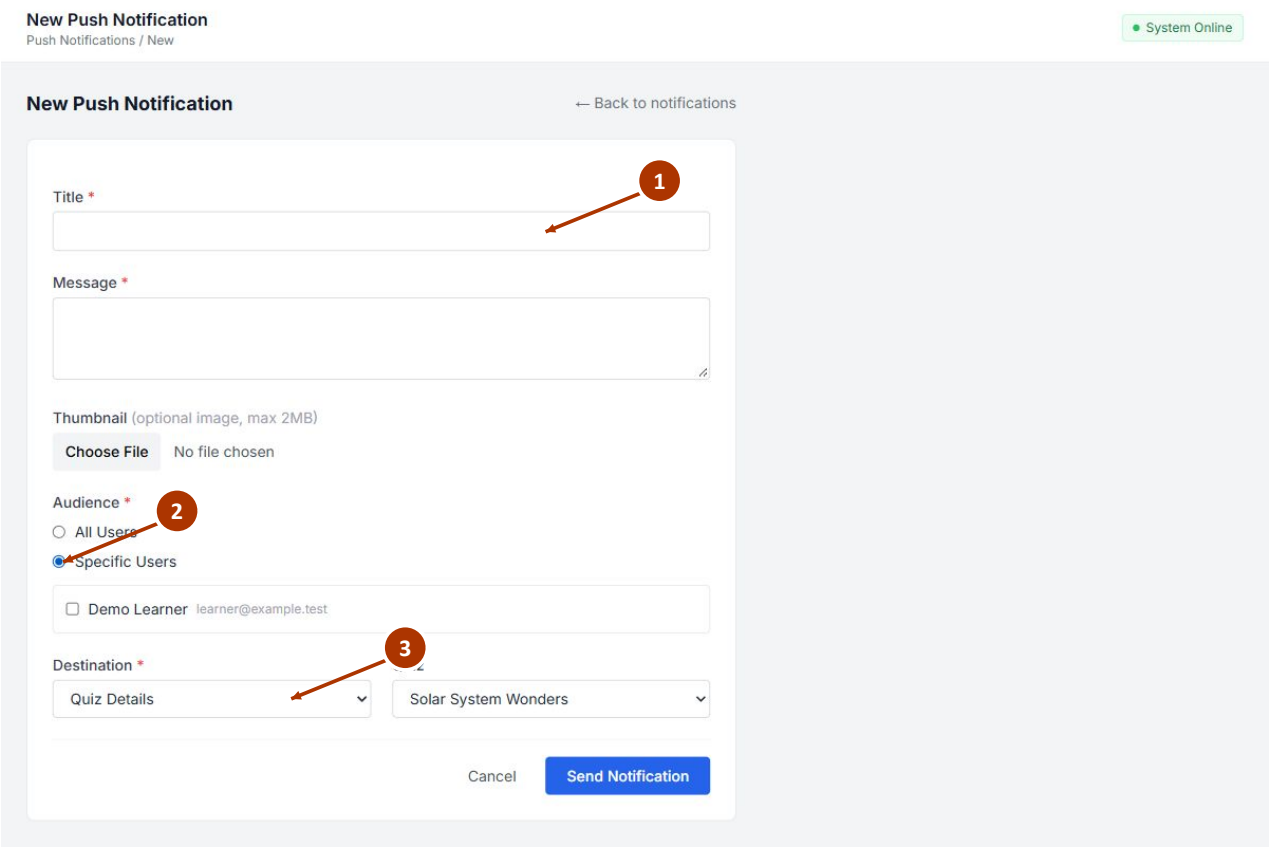
app/Models/User.php

app/Services/TargetService.php

Send notifications

Choose message content, audience, and tap destination. Sending creates an in-app inbox recipient record; phone push also requires FCM credentials, permission, and a registered device token.

Screen / location: Admin → Push Notifications → New Push Notification



Actual source preview - local demo data - live values may differ

1. Title, message, and image

2. All or Specific Users

3. Destination and quiz

Step by step

1. Open New Push Notification. Enter a concise Title and a clear Message.

2. Optionally upload a Thumbnail image up to 2 MB.

3. Choose All Users or Specific Users. Specific requires at least one selection; verify a test message with one intended account first.

4. Select Destination. Quiz Details additionally requires a valid Quiz selection.

5. Check spelling, audience, and destination, then click Send Notification.

6. Open List → View to inspect recipients and read state. Check the app inbox and native notification tap separately.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Title	Required string; maximum 255 characters.	Notification heading.
Message	Required string; maximum 1000 characters.	Inbox and push body.
Thumbnail	Optional image; maximum 2048 KB.	Notification image URL; rendering depends on the device OS.
Audience	all / specific. Eligible users are active non-admin accounts; this scope does not filter by email verification.	Creates one recipient row per resolved user. Do not test All Users using an administrator account.
Destination	none / attendance / quiz_details / target_progress / achievement / profile.	Controls the app screen opened on tap.
Quiz	Existing quiz ID required for Quiz Details.	Opens the question flow; unavailable quizzes fall back to categories.

Data and behavior in detail

Actual destination behavior

None: announcement without further navigation. Attendance: attendance screen. Quiz Details: selected quiz's question flow. Daily Target: Profile, not a separate target route. Achievement: achievements screen. Profile: profile screen.

Sent versus Read

Status becomes sent when the service starts; it does not prove phone delivery. Recipient read_at changes when the app marks a message read. This version does not persist reliable per-device delivery/failure counts. Inbox records may exist even when Firebase is absent and sending is only logged.

Scheduling, editing, and retry

Current routes provide list/create/show/store. There are no scheduled-send, existing-message-edit, delivery-retry, or campaign-segmentation controls. Sending a correction creates a new notification.

Audience and destination: every choice

1. Enter Title and Message and optionally upload a thumbnail. Choose the all-users audience or the specific-users audience radio button.

2. Specific users reveals the user picker. Tick every intended recipient; switching to this audience without choosing recipients is incomplete. Check identities before sending.

3. Destination offers None (open notifications), Attendance, Quiz Details, Daily Target, Achievement, and Profile. None opens notifications; Attendance opens attendance; Achievement opens achievements; Profile opens the profile.

4. Quiz Details reveals the Quiz dropdown. Choose the saved quiz title that the message refers to. This destination needs a quiz identifier; a missing or invalid quiz cannot open the intended quiz details.

5. Daily Target currently opens the profile area used for target progress, rather than a separate target screen.

6. Submit the send form once, then open the notification detail to review recipients and read state. Test a received notification on a configured device. A sent record alone does not prove that every device displayed the push.

Example: announce a quiz

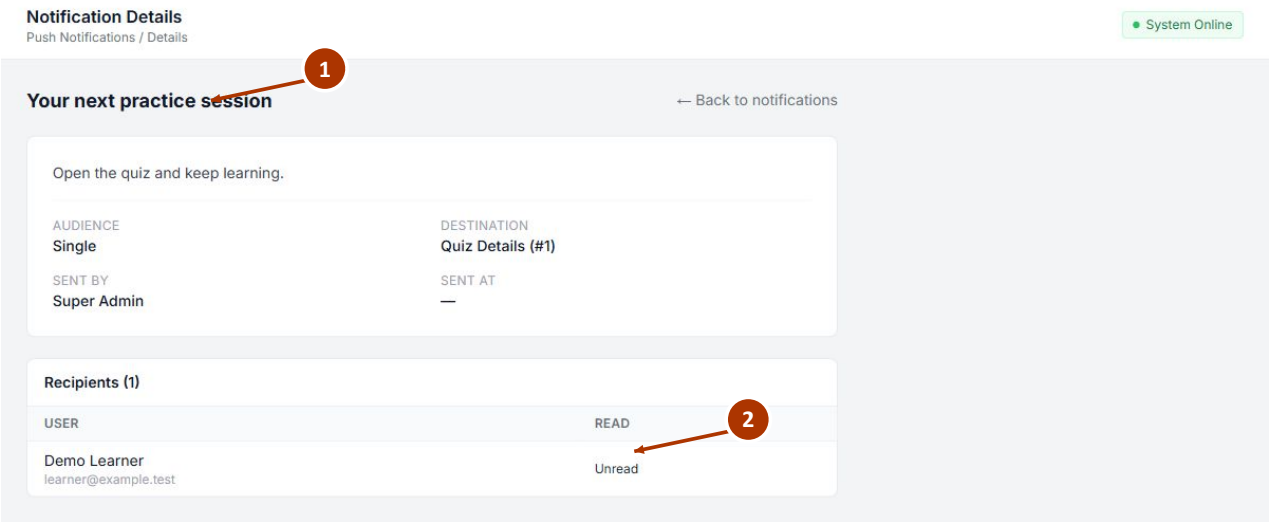
Title: **New practice quiz available**. Message: **Open today's mathematics quiz and try the new questions.** Choose your intended audience, select **Quiz Details**, and select the mathematics quiz in **Quiz**. Send, then tap the received notification and confirm the correct quiz opens. There is no scheduling, edit, or retry control in this form.

Verification after saving

Use the intended signed-in account on a real Android/iOS device with permission. Read the inbox message and confirm its Read timestamp in admin View. Browser preview does not validate native push delivery.

Recipients and read tracking

View shows content, audience, destination, and recipients. Unread means the inbox has not marked the message read; it does not prove a phone-delivery failure.



Actual source preview - local demo data - live values may differ

1. Message details

2. Recipient read state

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/PushNotificationController.php

app/Services/PushNotificationService.php

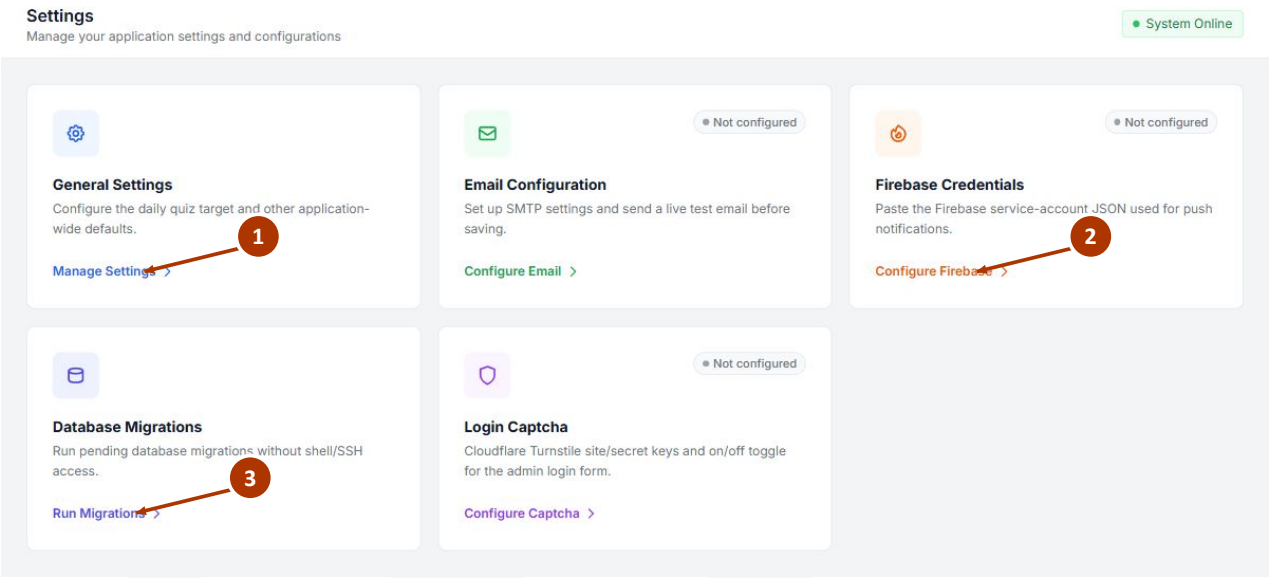
app/Services/FcmService.php

lib/services/notification_navigation.dart

Settings: every customization and its storage

Settings contains five tools: General, Email, Firebase, Database Migrations, and Login Captcha. This chapter maps each tool to its saved values. For server-only options and editing instructions, open the Environment variables & editing guide.

Screen / location: Admin → Settings



Actual source preview - local demo data - live values may differ

1. General and Email

2. Firebase

3. Migrations and Captcha

Step by step

1. Open Settings and choose the required tool.

2. Use General for global targets, Email for mail transport, and Firebase for device push.

3. Use Database Migrations for schema changes after a code release and Login Captcha for administrator login protection.

4. Read the dedicated chapter and verify through the actual feature after saving.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
General	Global Daily Target from 1-1000; stored in app_settings.global_daily_quiz_target.	Default for new daily rows. Individual custom targets override it; DEFAULT_DAILY_QUIZ_TARGET is only the unsaved fallback.
Email	Host, port, username, password, From address/name. Saves MAIL_* variables and sets MAIL_MAILER=smtp.	Transport for OTP, account, and target emails. Send Test uses current fields without saving; queued mail still requires a worker.
Firebase	Service-account JSON; Verify & Save writes FCM_PROJECT_ID and FCM_CREDENTIALS_JSON.	Authorizes server push. Does not replace native Firebase project files or grant device permission.
Database Migrations	Pending migration status and execution; no appearance or environment-value form.	Applies schema changes. Does not reset existing business records or update the app binary.
Login Captcha	Enable, Site Key, Secret Key; saves TURNSTILE_ENABLED / TURNSTILE_SITE_KEY / TURNSTILE_SECRET_KEY.	Administrator-login bot verification; not an app-wide notification or language preference.
Additional server customization	XP, levels, heartbeat, online timeout, result thresholds, fallback language, offerwall, queue/cron, and infrastructure.	Configured through exact variables in the environment reference; these are not hidden sliders in the five Settings tools.
App appearance and integrations	Branding, UI text, colors, API base URL, ad-unit IDs, and countdown are app source/platform files.	Require code/assets and a new app build; backend .env cannot change them automatically.

Data and behavior in detail

Admin save versus manual environment editing

Email, Firebase, and Captcha forms use EnvFileWriter to back up the old .env and clear configuration cache. A manual edit needs its own backup and config refresh. A later form save can overwrite the same variables. General is stored in the database and therefore takes precedence over the environment fallback.

How to choose the correct editor

Use the dedicated form when it exists, especially Firebase Verify & Save and Email Send Test. Use .env for server-only settings and advanced transport configuration. Read the step-by-step environment editing procedure and variable reference before applying a change.

Verification after saving

A Configured badge indicates configuration availability. Verify delivery, login, or schema behavior separately.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

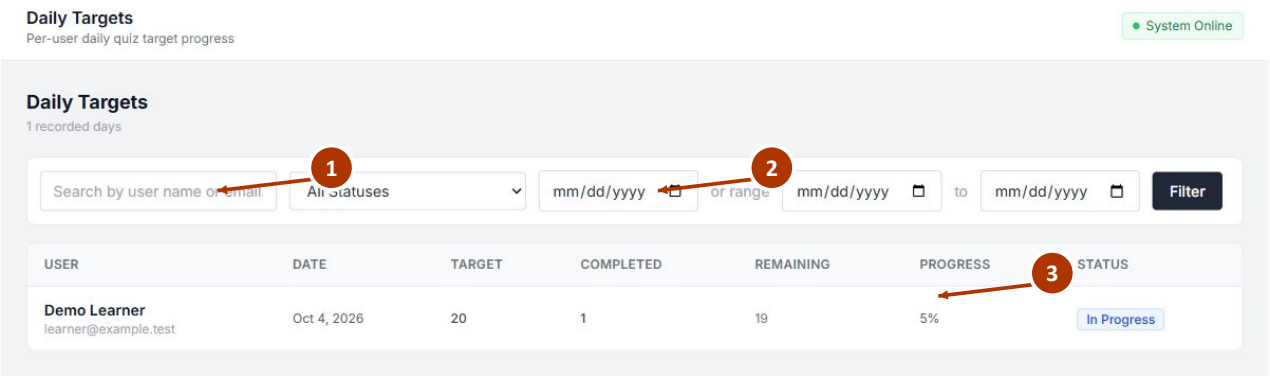
resources/views/admin/settings/index.blade.php

app/Http/Controllers/Admin/SettingsController.php

Read daily targets and history

Daily Targets is a reporting screen. Change target values in Settings → General or Users → Edit. This table shows the fixed target and completion status for each user and day.

Screen / location: Admin → Daily Targets



Actual source preview - local demo data - live values may differ

1. User and date filters

2. Target and completion progress

3. Daily status

Step by step

1. Search by user name/email and choose a status.

2. Use Exact date for a single day; it takes precedence over the From/To range.

3. Click Filter and read Target, Completed, Remaining, Progress, and Status.

4. Go to General Settings or Edit User to change target values; this table does not edit them.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Target	effective_target captured when the daily row is first created.	Priority: custom user target → saved global target → configuration fallback.
Completed	Count of quizzes reaching completed status.	Starting or abandoning a quiz does not increase it; completion is separate from passing.
Remaining	max(target − completed, 0).	Changes with the completed count.
Progress	min(100, completed ÷ target × 100).	Extra completions do not take the percentage above 100.
Status	Today: not_started / in_progress / completed. Older unfinished days may display not_completed.	Filtering only changes the report view, not the record.
Business date	business_timezone in config/quiz.php; Asia/Kolkata by default.	Daily boundaries use one business timezone for all users.

Current implementation limit
Changing the global target from 20 to 10 does not rewrite today's existing target of 20. A new daily row receives the new value, preserving history and earned XP.

Data and behavior in detail

Use target filters

1. Choose an exact date, or leave it empty and use a date range. An exact date takes priority over the range.

2. Status offers All Statuses, Completed, In Progress (today), Not Started (today), and Not Completed (past days). Choose the option appropriate to the date being reviewed.

3. Submit filters, then compare completed quiz count with that day's target. A target record keeps the value assigned when the daily row was created; changing the global setting does not rewrite that row.

4. To change future goals, use General Settings for the global value or edit the learner's custom target. Return to the app and verify a new applicable daily record.

Verification after saving

Complete one quiz and confirm the count increases once. The first target completion awards daily XP once; additional completions that day do not repeat the reward.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/TargetController.php

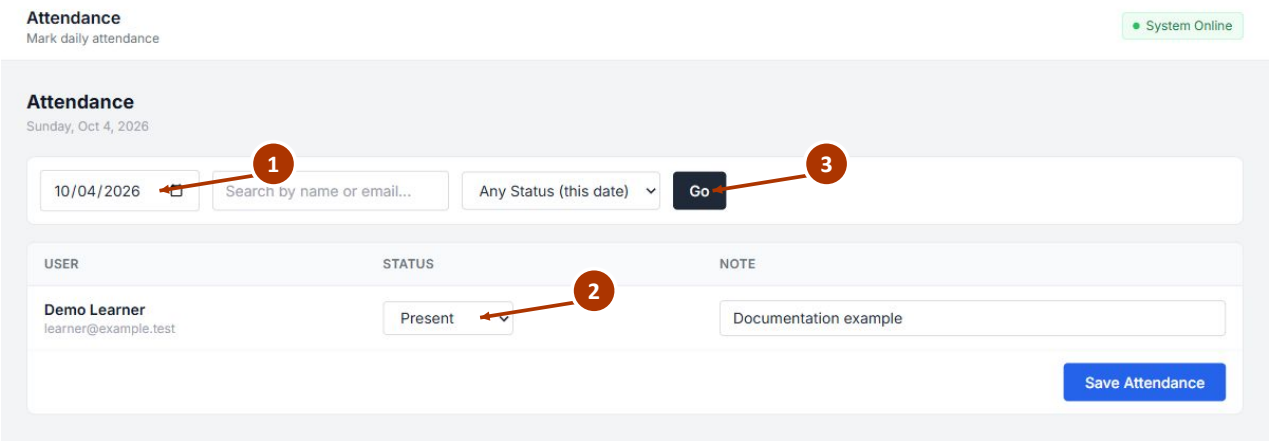
app/Services/TargetService.php

app/Services/ProgressionService.php

Mark attendance

Attendance is recorded by administrators separately from quiz playing. Present, Absent, and Leave entries belong to a user and date and appear in app attendance views.

Screen / location: Admin → Attendance



Actual source preview - local demo data - live values may differ

1. Date and filters

2. User status and note

3. Save attendance

Step by step

1. Choose a date and click Go. Use search/status filters to display the intended users.

2. Select Present, Absent, or Leave for each displayed user and optionally enter a note up to 255 characters.

3. Click Save Attendance. The server updates or creates records by user_id and date.

4. A changed status creates an attendance-destination notification for the affected user. A note-only change does not create a status-change notification.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Date	Required date for the selected records.	Creates an entry for that day in app history.
Status	present / absent / leave; blank means unmarked.	Replaces an existing status for that date. A blank selection is not a delete operation.
Note	Optional text, maximum 255 characters.	Saved with the record and can be visible in the app; avoid private internal notes.
Save	Processes submitted valid user statuses.	Users hidden by filters are not automatically updated.

Data and behavior in detail

Select and save attendance

1. Select the attendance date and locate the learner. The filter dropdown offers Any Status (this date), Present, Absent, Leave, and Not Marked.

2. Each learner row offers Not marked, Present, Absent, and Leave. Choose the actual status and add a note if needed, then submit the attendance form.

3. Not marked skips that row; it does not delete an existing attendance record. A blank day is not automatically Absent.

4. Reload the same date to verify the saved status, then open the learner's Attendance screen. A status change triggers the configured notification behavior; changing only a note does not send a status-change notification.

Verification after saving

Open app Attendance and compare date and status. Saving the same status again should not create another status-change notification.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/AttendanceController.php

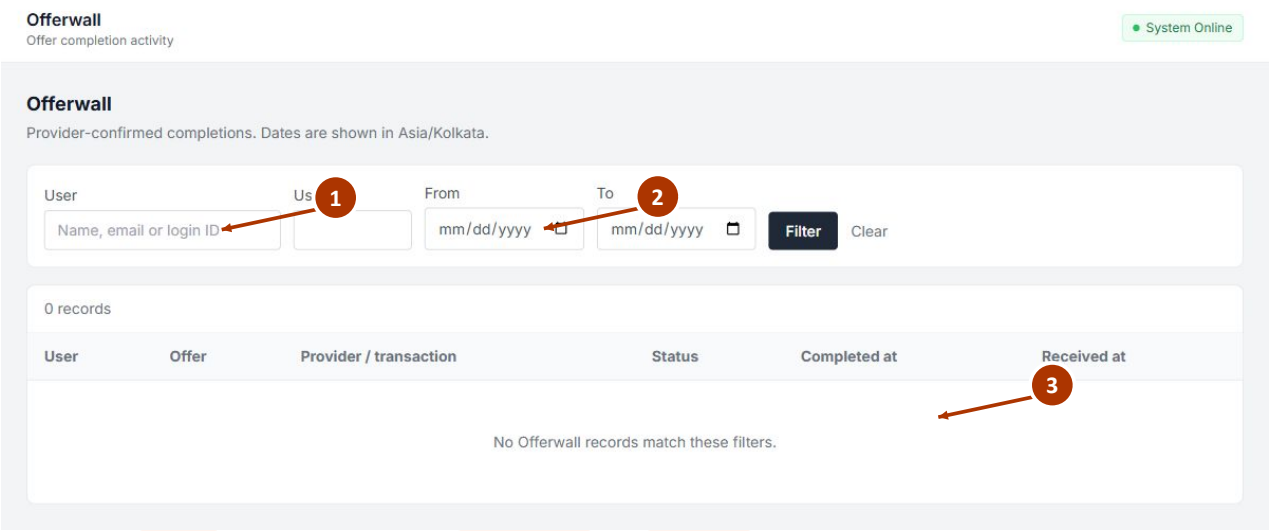
app/Services/AttendanceStatsService.php

lib/screens/attendance_screen.dart

Configure and inspect offerwall completions

The table shows provider-confirmed completions. Opening an offer and receiving its completion callback are different events. This page has no offer-creation or payout-settings form.

Screen / location: Admin → Offerwall



Actual source preview - local demo data - live values may differ

1. User search / ID

2. Completion date filters

3. Provider transactions

Step by step

1. Filter Offerwall by name/email/Login ID or User ID.

2. Choose From/To completion dates; To must not precede From.

3. Read User, Offer, Provider / transaction, Status, Completed at, and Received at. Use Clear to remove filters.

4. Configure OFFERWALL_ENABLED, OFFERWALL_SDK_KEY, OFFERWALL_PLACEMENT, callback secret/mode, and the provider callback at /api/offerwall/callback/unity in server configuration.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
OFFERWALL_ENABLED	False by default; server configuration.	Controls launch API availability.
SDK_KEY / PLACEMENT	Provider settings; default placement #WebOfferwall.	Used to generate the Unity rewards launch link.
CALLBACK_SECRET / MODE	hmac_sha256 by default; verified by the server controller.	Only verified callbacks create completion records.
Transaction	Unique by provider and hashed transaction key.	Duplicate callbacks do not add another completion; the same transaction is not accepted for another user.
Completed / Received	Provider completion time and server receipt time.	Used for reporting/filtering, separately from app-open time.

Current implementation limit
Current OfferwallService records completions. It does not automatically credit a wallet, create payouts, or increase quiz score/XP. Ad mediation settings are separate.

Verification after saving

Test an actual provider completion callback. Opening the offerwall is not completion evidence. The local capture is empty because provider credentials are unavailable.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

config/offerwall.php

app/Http/Controllers/Api/OfferwallController.php

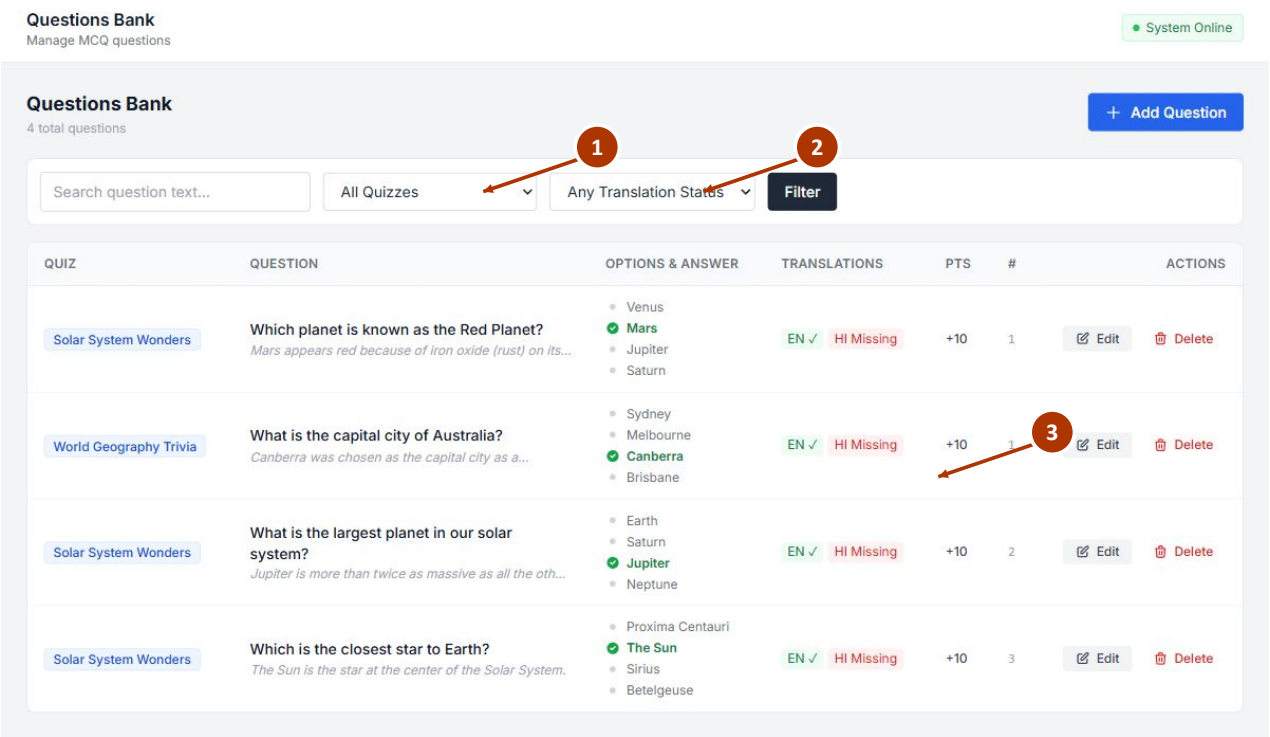
app/Services/OfferwallService.php

lib/services/offerwall_service.dart

Search and filter administrator lists

Lists help find the correct record before editing. Search and filters change the displayed results, not the stored business data.

Screen / location: Admin → Categories / Quizzes / Questions / Users



Actual source preview - local demo data - live values may differ

1. Quiz / text search

2. Translation filters

3. Edit and actions

Step by step

1. Open the relevant list and enter the record's known identifier or text.

2. Select filters appropriate to that list, then apply them.

3. Open the correct row's Edit or View action after checking its identity.

4. Use Clear to remove filters and verify the full list again.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Categories list	Name, slug, quiz count, order/active, and actions.	Create via Add Category; edit values in the Category form.
Quizzes list	category_id and text search.	Check question count and active state before editing a filtered quiz.
Questions Bank	quiz_id, text search, and translation_status.	Missing English/Hindi filters locate incomplete bilingual content.
Users list	Role, search, online_status, date_from/date_to.	Search name/email/Login ID; filter role, heartbeat online state, and registration dates.
Quiz Attempts	User, category, quiz, status, and date range.	Summary export retains query filters.
AJAX loading	admin-ajax-table.js updates some lists without full reload.	A spinner/skeleton indicates a pending response, not a final zero-record result.

Data and behavior in detail

What the delete popup looks like

Delete actions use the browser's native confirmation dialog, not a branded modal. It displays the row-specific message and typically **OK** and **Cancel**; wording and appearance can vary with browser language. Cancel closes the dialog without sending deletion. OK submits the request. The table refreshes after a successful response. If you see **Delete failed. Please try again.**, dismiss the alert, reload, and confirm the current row state before attempting again.

Use the relevant chapter's exact confirmation message to recognize the action. There is no undo button in this confirmation flow. A confirmation dialog is not a success message: wait for the refreshed table before assuming the action completed.

When an option or record is missing

1. Clear search and status filters, then reload the list.

2. Check that the parent record exists: a quiz needs a category; a question or Quiz Details notification needs a quiz.

3. Save the parent record first, return to the dependent form, and reload its dropdown.

4. If saving fails, read the validation message, correct the named field, and resubmit. Do not assume a row was created because the form was opened.

Verification after saving

Find a known record, Clear filters, and compare again. An empty filtered list does not mean the production records were deleted.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/QuestionController.php

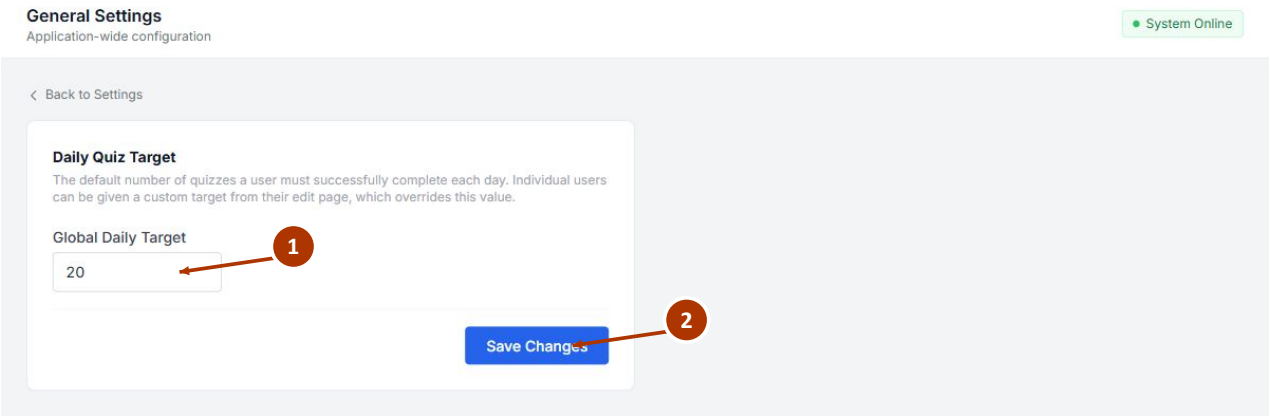
app/Http/Controllers/Admin/UserController.php

public/js/admin-ajax-table.js

Set the global daily target

Global Daily Target is the default number of quizzes per day. A custom user target takes priority.

Screen / location: Settings → General Settings



Actual source preview - local demo data - live values may differ

1. Target from 1-1000

2. Save Changes

Step by step

1. Enter an integer from 1-1000.

2. Click Save Changes to store app_settings.global_daily_quiz_target.

3. Users without custom targets receive this default when new daily records are created.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Global Daily Target	Saved setting; configuration fallback is 20 before the first save.	Changes effective targets in new daily reports and profile progress.

Current implementation limit
Existing history is not rewritten. If today's row already has target 20 and you save 5, that row remains 20.

Verification after saving

Check a new business date's progress for an account without a custom target.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

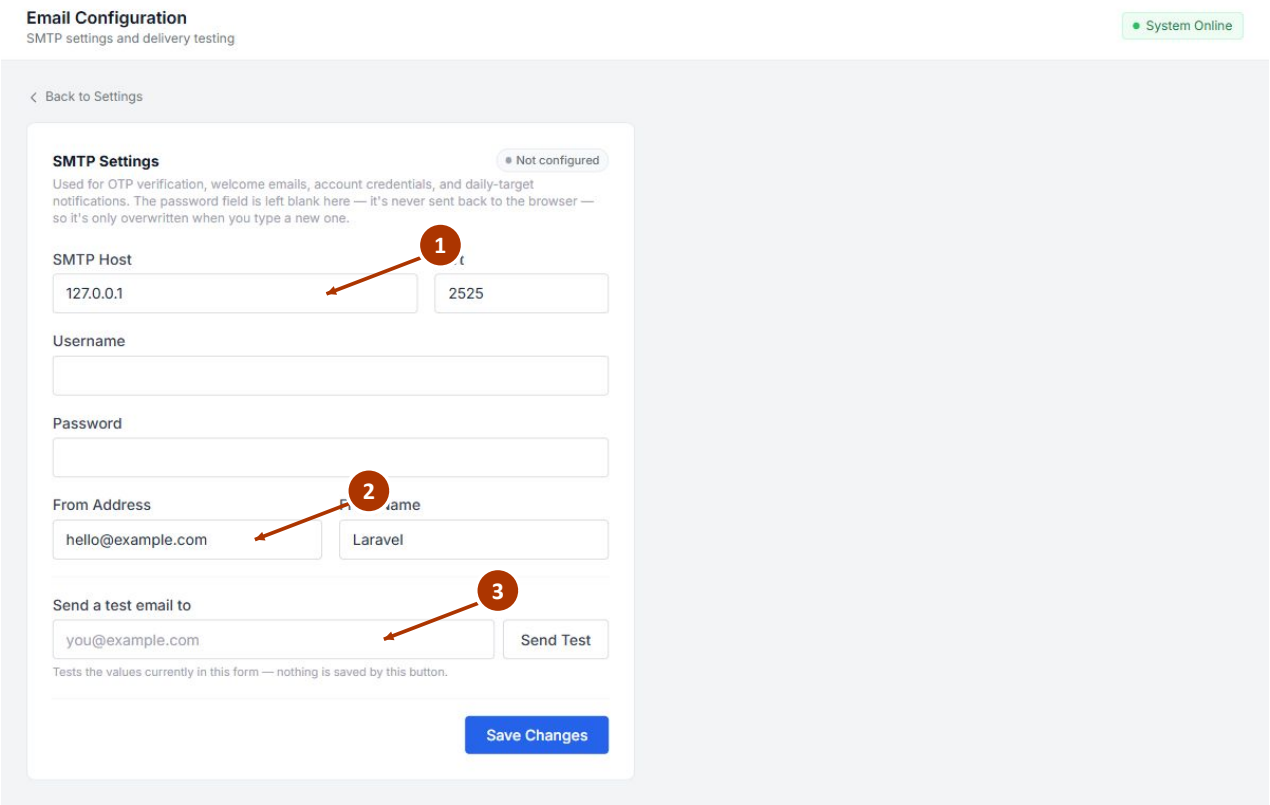
app/Services/TargetService.php

app/Models/AppSetting.php

Configure email and OTP delivery

SMTP transports verification OTPs, account details, and daily-target mail. Send Test and Save Changes are separate actions.

Screen / location: Settings → Email Configuration



Actual source preview - local demo data - live values may differ

1. SMTP host and port

2. Sender identity

3. Test recipient and save

Step by step

1. Get Host, Port, Username, and Password from your SMTP provider. Enter From Address and From Name.

2. Enter a recipient in Send a test email to and click Send Test. This tests the current form values without saving them.

3. After a successful test, click Save Changes. Stored passwords are not shown; leaving Password blank retains the existing password.

4. Run the queue worker or configured cron queue runner for queued mail. Test OTP/account flows separately end to end.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
SMTP Host / Port	Required host; integer port from 1-65535.	Selects the mail transport endpoint.
Username / Password	SMTP credentials; password required for initial configuration.	Provider authentication. Keep credentials in private server configuration.
From Address / Name	Valid sender email and display name.	Identity shown to recipients.
Test recipient	Valid email; used by Send Test only.	Sends a live email without saving settings.

Current implementation limit
Email settings do not configure phone push. A successful SMTP test does not guarantee queued emails will run when the worker is stopped.

Data and behavior in detail

Complete the email form and check the result

1. Enter the SMTP host, port, username, sender address/name, and applicable scheme values for your provider.

2. Enter a password when adding or replacing credentials. A blank password retains the existing password.

3. Save and read the validation or result message. The form writes the corresponding backend environment values and clears configuration cache.

4. Refresh cached production configuration and restart long-running workers where applicable. Verify a real application email with an account you control before using Reset Password for another learner.

Verification after saving

Check inbox/spam for the test, then check OTP/account delivery and queue processing.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/EmailSettingsController.php

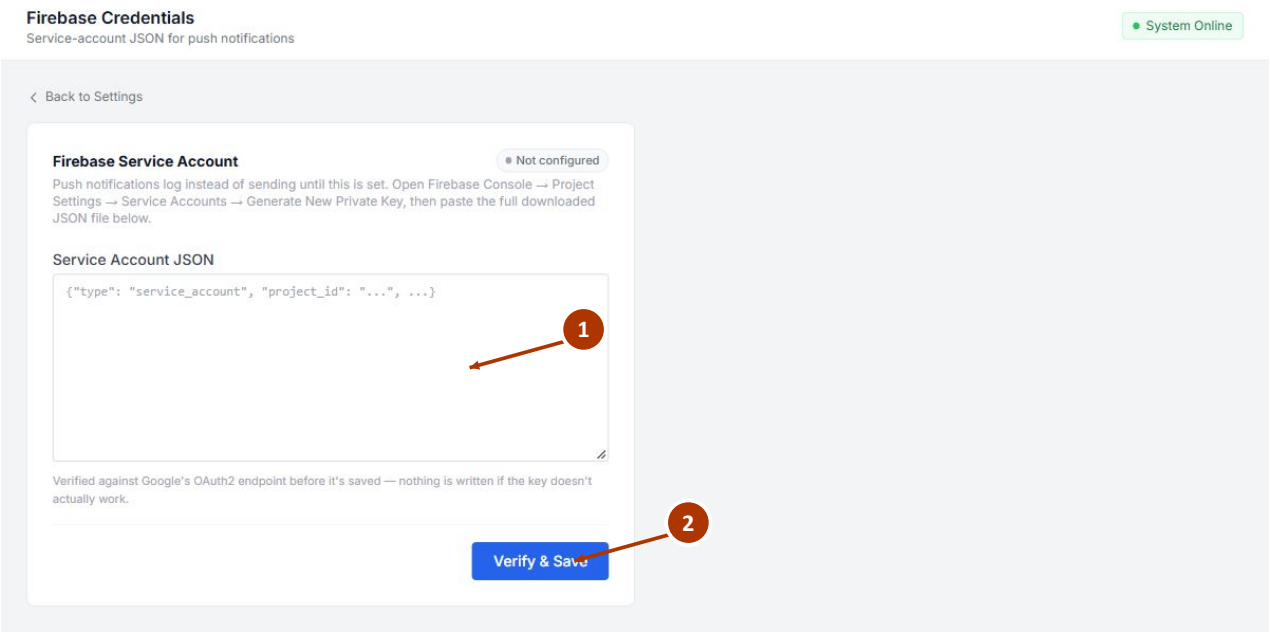
config/mail.php

app/Http/Controllers/CronController.php

Configure Firebase push credentials

The server service account and Flutter Firebase app configuration are separate. Both must use the intended Firebase project.

Screen / location: Settings → Firebase Credentials



Actual source preview - local demo data - live values may differ

1. Service Account JSON

2. Verify & Save

Step by step

1. Obtain the complete service-account private-key JSON for your Firebase project.

2. Paste it into Service Account JSON. Keep this private credential out of public documentation and mobile assets.

3. Click Verify & Save. The server saves only after JSON validation and Google OAuth verification succeed.

4. Check native app configuration, notification permission, and token registration for the same project.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Service-account JSON	type service_account and required project, private-key, and client fields.	Authorizes server FCM HTTP requests.
Verify & Save	Rejected Google verification prevents saving.	Correct invalid JSON or key errors before retrying.
Device token	Registered by the app through POST /device/fcm-token.	Required to send push to that user's devices.

Data and behavior in detail

Paste, save, and verify credentials

1. Copy the intended Firebase project's service-account JSON into the credentials form; do not paste a web configuration object or only an API key.

2. Submit the form and resolve any verification error before continuing. Successful verification saves the push project and JSON configuration.

3. Confirm the app uses the intended Firebase project and that a test device has registered its token. Send one specific-user notification and test its tap destination.

4. Check delivery independently from the Configured badge. Authentication FIREBASE_API_KEY and push service-account credentials have different purposes.

Verification after saving

Sign in on a native device, grant permission, send a Specific Users message, and check both the system notification and inbox. No private key was entered in the local capture.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/FirebaseSettingsController.php

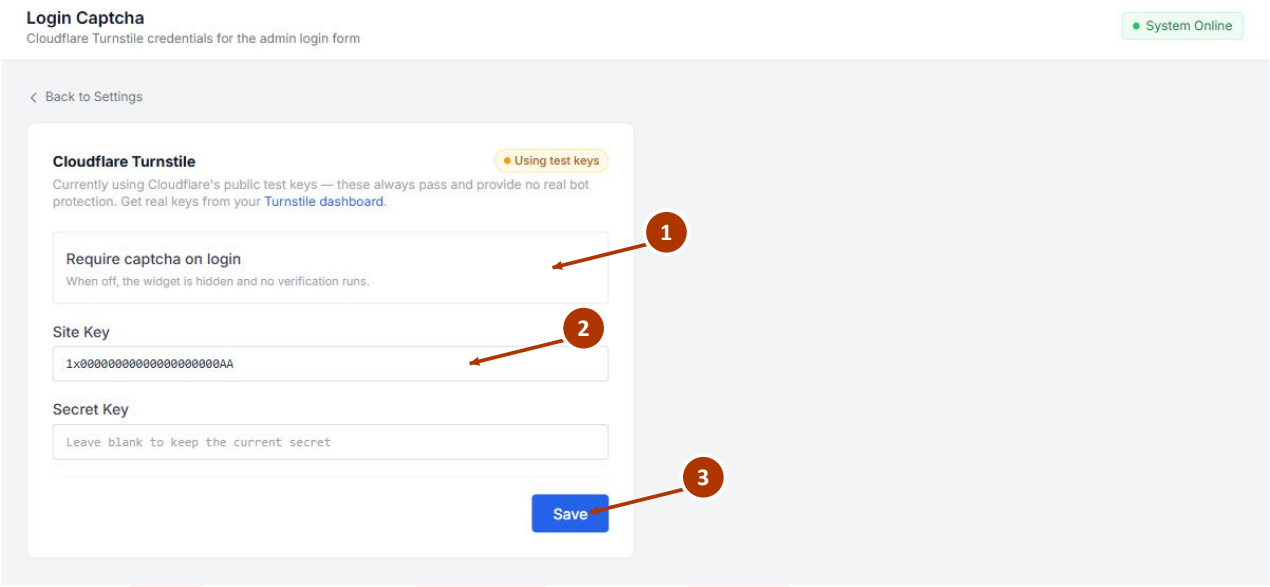
app/Services/FcmService.php

lib/services/push_notification_service.dart

Configure administrator login CAPTCHA

Turnstile protects administrator login. The Site Key renders the browser widget; the Secret Key verifies challenges on the server.

Screen / location: Settings → Login Captcha



Actual source preview - local demo data - live values may differ

1. Enable switch

2. Site and secret keys

3. Save

Step by step

1. Create Turnstile keys for your administrator domain.

2. Enter Site Key and, for first configuration, Secret Key. Enable Require captcha on login.

3. Save and verify the widget and challenge on the next login.

4. An existing secret is not shown in the form. A blank replacement field retains it.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Site Key	Required, maximum 255; public widget key.	Controls the login challenge widget.
Secret Key	Optional replacement, maximum 255.	Server challenge verification.
Require captcha	TURNSTILE_ENABLED switch.	When off, the widget is hidden and verification skipped.

Current implementation limit
The capture contains public test keys, which do not prove real bot protection. Use real domain keys for production.

Data and behavior in detail

Enable or update Turnstile

1. Set the enabled state, enter the site key, and enter the matching secret key when adding or replacing credentials.

2. Leave the secret blank only when retaining its existing value. Save, then reload the administrator login page.

3. Verify the widget on the configured domain and test login. The site key is public widget configuration; the secret remains server-side.

Verification after saving

Test a fresh login with the correct domain/key combination before leaving the existing administrator session.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

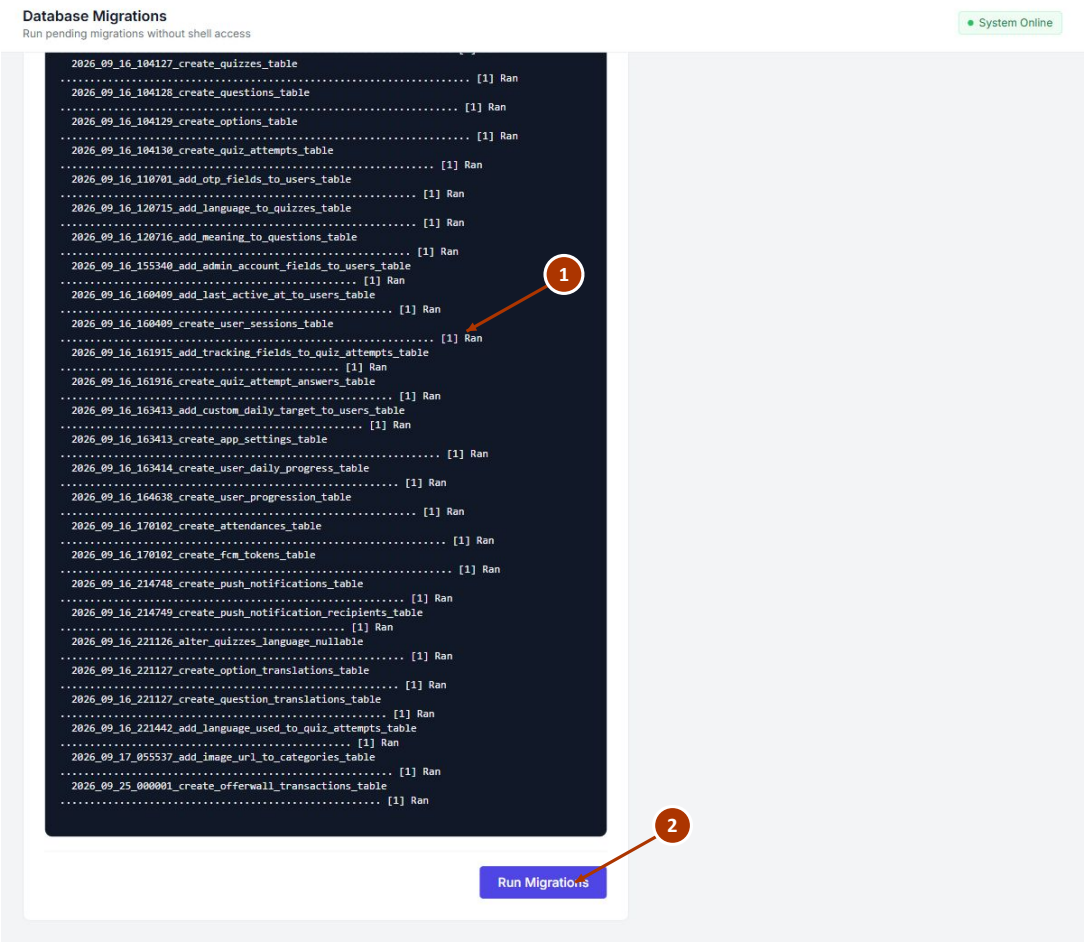
app/Http/Controllers/Admin/TurnstileSettingsController.php

app/Http/Controllers/Admin/AdminAuthController.php

Apply database migrations

This tool applies new tables and columns after a code release. It is not a question import, database restore, or appearance customization tool.

Screen / location: Settings → Database Migrations



Actual source preview - local demo data - live values may differ

1. Ran / Pending status

2. Run Migrations

Step by step

1. Back up the database and deploy the intended release files.

2. Open Database Migrations and read Ran/Pending in the migrate:status output.

3. Click Run Migrations for pending changes. Previously applied migrations are not rerun.

4. Verify related admin/API screens after success. Inspect logs and database credentials/schema if an error occurs.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Migration status	Migration filename, batch, and status.	Shows remaining schema updates.
Run Migrations	Applies pending server-side schema operations.	May add tables/columns; it is not a reset of business records.

Data and behavior in detail

Read the migration confirmation before proceeding

The run action asks **Run all pending migrations now? This changes the production database schema and cannot be undone by this button.** Choose **Cancel** to stop or **OK** only after checking the pending list and taking the required database backup. Review the resulting status and reload the pending list. This button is a database upgrade operation, not an option for editing quiz content.

Verification after saving

Confirm no pending migrations remain and the new feature does not fail with a missing-column error. This button does not update the mobile binary.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Admin/DatabaseMigrationController.php

database/migrations

Customize settings with environment variables

This is the configuration reference for the actual Quizs source. Edit the backend's private .env file, not the documentation files or Flutter assets. Some values are controlled by administrator forms; others are server-only configuration. The tables show current defaults and the effect of changes.

Screen / location: Backend project → .env → config/*.php

Step by step

1. Identify the backend serving the app. On cPanel, open File Manager, enable Show Hidden Files, and locate .env beside artisan and composer.json in the Laravel project directory. Locally, use Quiz Application Admin/.env. Do not confuse it with .env.example.

2. Make a private backup of .env before editing. Use .env.example only as a template for a new installation. Do not overwrite a live .env or replace its APP_KEY to change a feature.

3. Find the exact variable below. Replace its existing line, or add one line if absent. Keep one definition per key. Variables such as XP_PER_LEVEL are supported by config/quiz.php even though .env.example does not list them.

4. Use KEY=value syntax. Use true/false for booleans, plain integers for counts, and quotes around values containing spaces or a # character. For example OFFERWALL_PLACEMENT="#WebOfferwall". Preserve JSON quoting and private-key formatting; the verified Firebase admin form is safer than manually pasting JSON.

5. Save, then run php artisan config:clear from the backend directory. For a production deployment that uses cached configuration, run php artisan config:cache afterward. Restart long-running queue workers with php artisan queue:restart, or restart the relevant managed process so it loads the new configuration.

6. Reload the relevant administrator page and app flow. Sign in again when checking a changed heartbeat interval. Compare an actual result or delivery event, rather than relying only on a Configured badge.

7. If a change fails, restore the private backup, rebuild configuration cache, restart workers, and inspect storage/logs/laravel.log. Keep passwords and service-account keys out of public screenshots and this guide.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
APP_NAME / APP_URL	Application name and backend URL. .env.example uses Laravel and http://localhost.	Use your product name and final HTTPS domain. URL changes affect generated links/artwork; Flutter ApiClient.baseUrl must be changed separately and rebuilt.
APP_ENV / APP_DEBUG	Template: local / true. Production: production / false.	Controls environment and debug output, not app content or colors.
APP_KEY	Installation-specific encryption key.	Retain the existing key on upgrades. Generating another key is not a customization step.
APP_TIMEZONE / BUSINESS_TIMEZONE	Both default to Asia/Kolkata. Business timezone falls back to APP_TIMEZONE.	BUSINESS_TIMEZONE controls today/month boundaries in targets and statistics. Changing it can change which date activity belongs to; it does not rewrite historical daily rows.
APP_LOCALE / APP_FALLBACK_LOCALE	Template: en / en; Laravel application locales.	Do not control Flutter interface language or bilingual question fallback. Those have separate settings.
DB_CONNECTION / DB_HOST / DB_PORT / DB_DATABASE / DB_USERNAME / DB_PASSWORD	Template uses sqlite; hosted MySQL needs its own connection values.	Selects the application database. Changing credentials does not migrate old records into another database.
DEFAULT_DAILY_QUIZ_TARGET	Default 20; fallback in config/quiz.php.	Used only if no global target has been saved in app_settings and the user has no custom target. It does not override the General Settings value.
XP_PER_COMPLETED_DAILY_TARGET	Default 10; integer reward in config/quiz.php.	Changes XP awarded for future first daily-target completions. Does not retroactively add XP to previously completed days. Use a nonnegative product-approved value.
XP_PER_LEVEL	Default 100; LevelService uses at least 1.	Computed level = floor(total XP ÷ XP_PER_LEVEL) + 1. Changing the curve can alter derived levels without changing earned XP; stored current_level used by ranking is not bulk-recalculated by editing .env.
HEARTBEAT_INTERVAL_SECONDS	Default 45; sent in authentication responses.	Changes the interval the app receives at login. Existing app state may retain its old interval until sign-in/reinitialization.

Field / Feature	Definition and source	Effect of changes
ONLINE_TIMEOUT_SECONDS	Default 120.	Controls when last_active_at is considered online. Keep it longer than the expected heartbeat interval to avoid intermittent offline estimates.
ACTIVE_USERS_WINDOW_DAYS	Default 30.	Rolling active-user count window; not the separate Active This Month calendar count.
PERFORMANCE_EXCELLENT_THRESHOLD	Default 90; percent correct out of all quiz questions.	Selects excellent result classification; not the per-quiz passing threshold.
PERFORMANCE_GOOD_THRESHOLD	Default 70.	Selects good classification below excellent.
PERFORMANCE_AVERAGE_THRESHOLD	Default 50; below this is low.	Keep thresholds ordered excellent ≥ good ≥ average within 0-100. Display wording is in Flutter results_screen.dart.
FALLBACK_LANGUAGE	Default en; supported question languages are en and hi.	Fallback translation for bilingual questions when requested content is missing. Does not translate text or change the app's interface-language preference.
MAIL_MAILER	Template/default log; Email Configuration Save sets smtp.	log writes mail to logs instead of sending to recipients. Set smtp for configured SMTP delivery.
MAIL_HOST / MAIL_PORT	Defaults 127.0.0.1 / 2525, not universal provider values.	Use the actual provider endpoint and port. Email Configuration writes these variables.
MAIL_USERNAME / MAIL_PASSWORD	Template null; private SMTP credentials.	Email Configuration writes username and a supplied replacement password. Blank password in the admin form retains the stored one.
MAIL_FROM_ADDRESS / MAIL_FROM_NAME	Template hello@example.com / APP_NAME.	Sender email/name for recipients. Configure a provider-authorized sender; the admin form writes these values.
MAIL_SCHEME / MAIL_URL / MAIL_EHLO_DOMAIN	Optional SMTP scheme, transport URL, and greeting domain from config/mail.php.	Advanced server-only configuration. Match the provider's settings. A populated MAIL_URL can supply transport parameters; resolve conflicts with individual fields. This source does not read MAIL_ENCRYPTION.
FIREBASE_API_KEY	Blank in the template; read by API authentication.	Firebase authentication configuration, separate from FCM push credentials and native google-services.json.

Field / Feature	Definition and source	Effect of changes
FCM_PROJECT_ID	Blank in the template.	Push target project. Firebase Verify & Save writes the project_id from the supplied service-account JSON.
FCM_CREDENTIALS_JSON	Blank in the template; complete service-account JSON inline.	Preferred when nonempty. The Firebase admin form verifies with Google before writing it. Invalid nonempty JSON does not fall back to FCM_CREDENTIALS_PATH.
FCM_CREDENTIALS_PATH	Blank; private server file path to service-account JSON.	Used only when inline JSON is empty. Keep the file outside public/. Manual editing does not perform the admin form's Google verification.
TURNSTILE_ENABLED	Configuration default true; may be absent from .env.example.	Enables administrator login verification. Login Captcha writes true/false here.
TURNSTILE_SITE_KEY / TURNSTILE_SECRET_KEY	Configuration defaults are public test keys.	Use your real domain's keys for production. The admin form writes site key and a supplied secret; blank replacement secret retains the previous one.
QUEUE_CONNECTION	Template database.	Queued mail needs queue processing and database queue tables. Changing the driver does not move existing queued jobs.
CRON_SECRET	Blank in the template.	Private shared key for GET /api/cron/run-queue. Required when using the protected HTTP queue runner; not a replacement for SMTP credentials.
SESSION_DRIVER / SESSION_LIFETIME	Template database / 120 minutes.	Administrator session storage and lifetime. Database sessions need the session table; not a quiz-duration setting.
CACHE_STORE	Template database.	Cache backend; database cache requires its tables. Changing the driver is infrastructure configuration, not target editing.
OFFERWALL_ENABLED	Default false.	Enables the offerwall launch feature. Requires provider settings as well.
OFFERWALL_SDK_KEY / OFFERWALL_PLACEMENT	Blank SDK key; default placement #WebOfferwall.	Provider launch configuration. Quote the placement so # is preserved.
OFFERWALL_CALLBACK_SECRET / OFFERWALL_CALLBACK_MODE	Private secret; default mode hmac_sha256.	Verifies provider callbacks. Completion records do not automatically credit wallet, quiz points, or XP.

Field / Feature	Definition and source	Effect of changes
FILESYSTEM_DISK / AWS_*	Template local, plus optional AWS storage fields.	Category uploads explicitly use the category_images disk. Changing the default disk alone does not move existing uploads or make that flow use S3. Configure the named disk separately if changing its storage.
LOG_CHANNEL / LOG_LEVEL	Template stack / debug.	Logging configuration. Does not change reports, notifications, or app strings. Protect server logs from public access.

Current implementation limit
All values are configuration guidance; this guide does not edit your running server. Keep credentials in the private backend .env and preserve existing application keys. Product defaults in the current source are starting values, not a promise of a finalized XP economy.

Data and behavior in detail

Which settings are saved in the database?

Settings → General stores app_settings.global_daily_quiz_target. Users → Edit stores custom_daily_target on the user record. Priority is individual custom target → saved global target → DEFAULT_DAILY_QUIZ_TARGET. The daily row then freezes that effective target. Example: .env fallback 5, saved global 20, and user override 8 means that user gets 8 for a new day; a global user gets 20.

Which admin forms write .env?

Email Configuration writes MAIL_MAILER=smtp, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_FROM_ADDRESS, MAIL_FROM_NAME, and a nonblank replacement MAIL_PASSWORD. Firebase writes FCM_PROJECT_ID and FCM_CREDENTIALS_JSON after verification. Login Captcha writes TURNSTILE_SITE_KEY, TURNSTILE_ENABLED, and a nonblank replacement TURNSTILE_SECRET_KEY. Admin save and manual .env editing change the same values; a later admin save can replace your manual edits.

Automatic backups and configuration refresh

Those three forms use EnvFileWriter: it backs up the previous nonempty .env in storage/app/private/env-backups, keeps up to 20 backups, writes the file, and runs config:clear. It does not automatically rebuild config cache or restart long-running workers. Manual File Manager editing does not create that automatic backup or clear the cache.

Example: change future XP and result classification

```
XP_PER_COMPLETED_DAILY_TARGET=15
XP_PER_LEVEL=150
PERFORMANCE_EXCELLENT_THRESHOLD=95
PERFORMANCE_GOOD_THRESHOLD=75
PERFORMANCE_AVERAGE_THRESHOLD=50
```

These are sample customization values, not new product defaults. Existing earned XP remains unchanged; inspect derived level and stored ranking level separately after a curve change. Test an attempt around each result threshold.

Example: configure SMTP using .env

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.your-provider.example
MAIL_PORT=587
MAIL_USERNAME="YOUR_SMTP_USERNAME"
MAIL_PASSWORD="YOUR_PRIVATE_SMTP_PASSWORD"
MAIL_FROM_ADDRESS="no-reply@your-domain.example"
MAIL_FROM_NAME="Quizs"
```

Replace placeholders with provider values. Port 587 here is an example, not a requirement. Configure MAIL_SCHEME if your provider requires it; this project uses MAIL_SCHEME, not MAIL_ENCRYPTION. After cache refresh, use Email Configuration → Send Test and test queued OTP/account mail separately.

Example: use a private Firebase credentials file

```
FCM_PROJECT_ID="YOUR_FIREBASE_PROJECT_ID"
FCM_CREDENTIALS_JSON=
FCM_CREDENTIALS_PATH="/home/account/private/firebase-service-account.json"
```

Replace the example path with an existing readable private file containing valid JSON. Empty inline JSON permits the file fallback. Confirm that the native app uses the same project, registers a token, and has permission.

Apply changes from Terminal or SSH

```
php artisan config:clear
php artisan config:cache
php artisan queue:restart
```

Run from the backend directory containing artisan. config:cache is appropriate when production uses cached configuration. queue:restart asks persistent workers to restart after their current job; a process supervisor must start them again. A one-shot HTTP cron worker starts fresh for each invocation. If you have no terminal, arrange equivalent deployment commands through the host's supported tools; saving a manual .env edit alone does not invalidate an existing configuration cache.

What cannot be customized through these variables?

Category/question content is managed in its forms. Quiz language is chosen at creation. Per-quiz passing percentage belongs to the quiz record. Ranking order is code in RankingService/QuizRankingService. App API URL, logo, theme, strings, ad-unit IDs, and the fixed Next Trial countdown are Flutter/platform source changes requiring a new build. Backend .env does not automatically become Flutter configuration.

Historical effects and verification

New target defaults do not rewrite existing daily rows. New reward values do not retroactively adjust earned XP. Editing thresholds does not automatically resubmit or rescore historical attempts. Changing XP_PER_LEVEL recalculates some derived level displays, while ranking reads stored current_level; a deliberate data migration is needed to make historical stored levels consistent. Do not invent a migration or reset production records just to test a setting.

Verification after saving

After each change, check its exact outcome: new-day target, one future XP award, login heartbeat/online timeout, boundary-case result classification, actual mail, real-device push, or a verified offerwall callback. If only documentation text changed, no application setting changed.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

.env.example

config/app.php

config/quiz.php

config/mail.php

config/services.php

config/offerwall.php

app/Support/EnvFileWriter.php

app/Services/TargetService.php

app/Services/LevelService.php

app/Services/ProgressionService.php

app/Services/RankingService.php

app/Services/FcmService.php

app/Http/Controllers/Admin/EmailSettingsController.php

app/Http/Controllers/Admin/FirebaseSettingsController.php

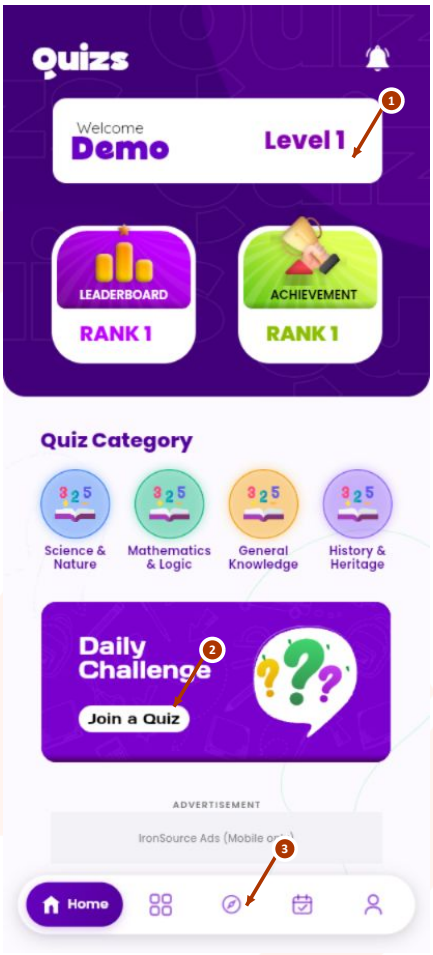
app/Http/Controllers/Admin/TurnstileSettingsController.php

lib/services/activity_service.dart

Walk through the app screens

These captures come from this Flutter source running as a local browser build. They show layout and API data; test native push, Google sign-in, ads, and Play updates separately on a real Android/iOS device.

Screen / location: Quizs App → Home / Quiz / Profile



Actual source preview - local demo data - live values may differ

1. Home content

2. Start a quiz

3. Navigation

Step by step

1. Sign in with email/Login ID and password. Google sign-in needs platform credentials. Sign Up and Forgot Password use OTP verification.

2. Choose a subject in Home / Categories and inspect quiz title, played count, and personal progress.

3. Open a quiz, select one answer, and complete next/submit flow. Next Trial currently uses a fixed 5-second countdown.

4. Read score, correct/wrong totals, performance, and rank in Results. Check daily target, XP/level, and activity statistics in Profile.

5. Read or tap Notifications and compare Attendance/Achievements with their server-backed reports.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Sign In / Sign Up / OTP	AuthService → /auth/login, /auth/register, /auth/send-otp, /auth/verify-otp.	Account controls in Users; SMTP for OTP; interface strings in app code.
Home / Categories	QuizApiService → /categories.	Active state, name, image, and order update on the next fetch.
Quiz listing	/categories/{id}/quizzes?language=…	Quiz title/active state, unique completed players, and current-user progress. Not every metadata field is shown in every view.
Question screen	/quizzes/{id}?lang=…; /start; /attempts/{id}/answer.	Text/options/points/translations from Questions Bank; countdown in question_screen.dart.
Results	/attempts/{id}/submit and quiz ranking endpoint.	Quiz score and answer statistics; performance thresholds differ from global ranking.
Profile / Edit Profile	/profile/stats and auth profile/avatar endpoints.	Identity, today's goal, XP/level, and activity; total user score is not progression XP.
Attendance	/attendance/today, /history, /summary.	Administrator-marked present/absent/leave, not automatic game attendance.
Achievements	/achievement and RankingService.	Level → XP → accuracy → completions → deterministic tie-break; quiz rank is separate.
Notifications	/notifications and recipient read / read-all endpoints.	Message content, destinations, and server read state.
Offerwall	POST /offerwall/launch.	Enabled/provider settings; opens an external offer URL.

Data and behavior in detail

Profile data sources

ProfileStatsService separately aggregates completed attempts and today's ongoing answers. Today's right/wrong, lifetime accuracy, current-month totals, daily target, and level/XP/rank come from this response. Attendance uses separate endpoints. Do not interpret a failed-load empty state as real progress.

Language, sound, and notification preference

Language uses AppLanguage's English/Hindi preference and may refetch bilingual questions. SoundService persists sound preference. Profile's Notification toggle is only local widget state: it does not change FCM subscriptions, OS permission, server opt-out, or survive restart. The Notification row separately opens the inbox.

Inbox filters and refresh

All, Quiz, and Rewards tabs filter by destination_type; Rewards represents achievement messages. Mark All as Read updates recipient records and rolls back optimistic state on failure. Pull to refresh loads new messages.

Text that does not come from admin

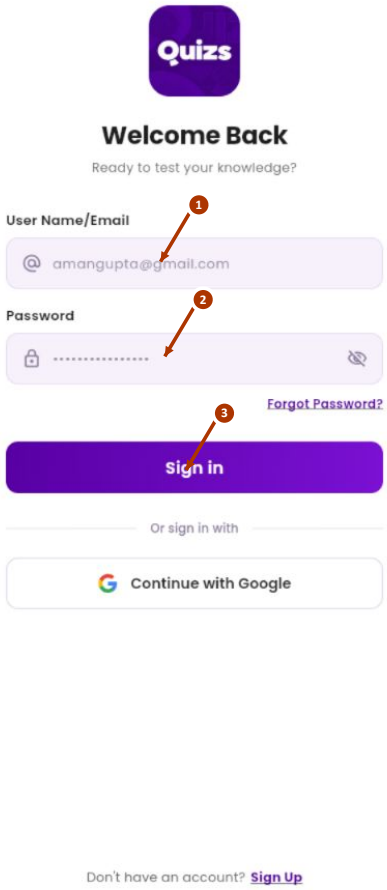
Buttons, menus, static headings, authentication copy, and interface translations are in lib/l10n/app_strings.dart and screen files. Logos, images, fonts, and widget colors are app assets/code. These changes require a new build.

Verification after saving

Run one complete category → quiz → answers → results → profile flow. Test native/provider integrations separately.

Sign In

Email/Login ID and password authenticate the account. Temporary accounts enter password change; Forgot Password starts OTP reset. Google requires native provider configuration.



Actual source preview - local demo data - live values may differ

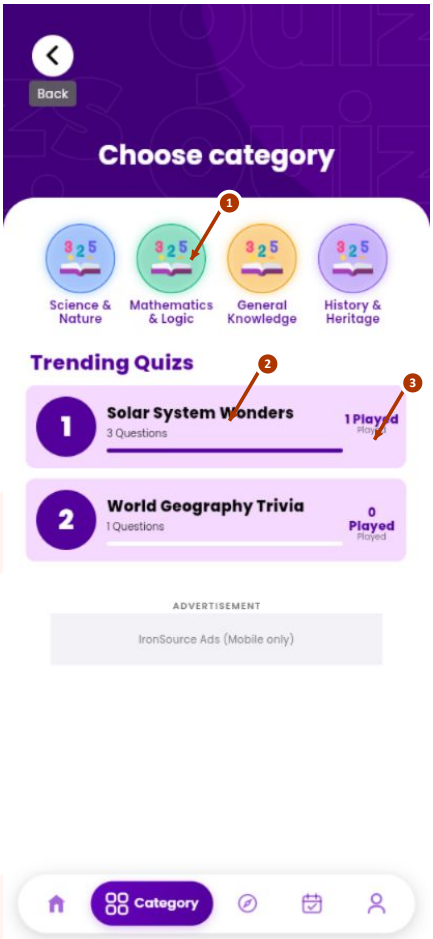
1. Login identity

2. Password

3. Sign in

Categories and quiz listing

Subject icons select a category. Quiz title and question count come from content; Played counts unique completed users, and progress is your question progress today.



Actual source preview - local demo data - live values may differ

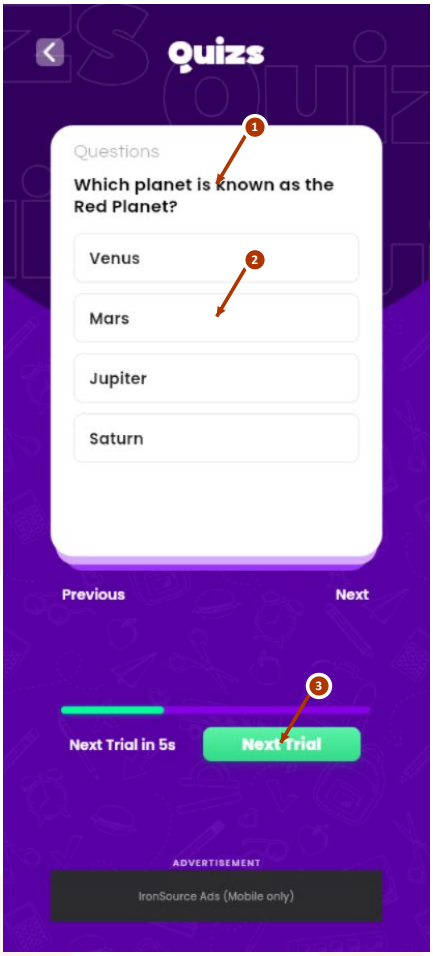
1. Choose category

2. Quiz title and questions

3. Played / progress

Choose an answer

Question text and four options come from admin content. Selecting an option displays evaluation and explanation; Previous/Next navigates questions.



Actual source preview - local demo data - live values may differ

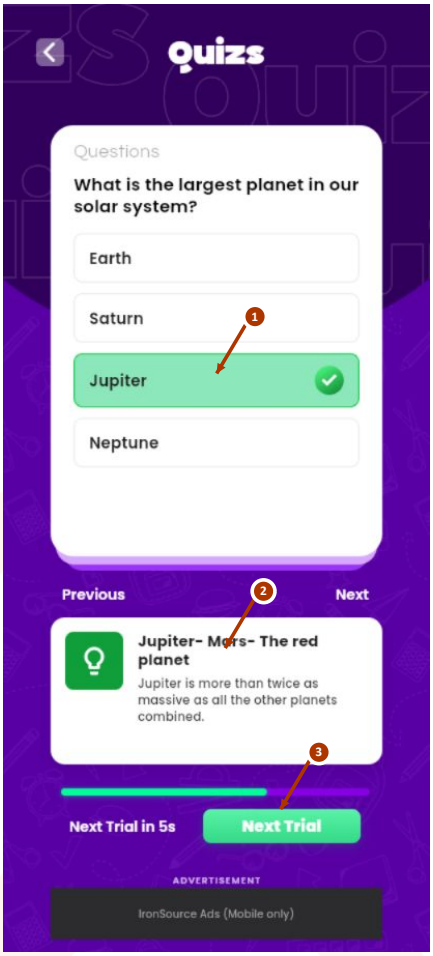
1. Question text

2. Choose one answer

3. Next Trial

Explanation after answering

The correct answer and Explanation use server content. Edit Explanation in Questions Bank to change the learning text shown on a later fetch.



Actual source preview - local demo data - live values may differ

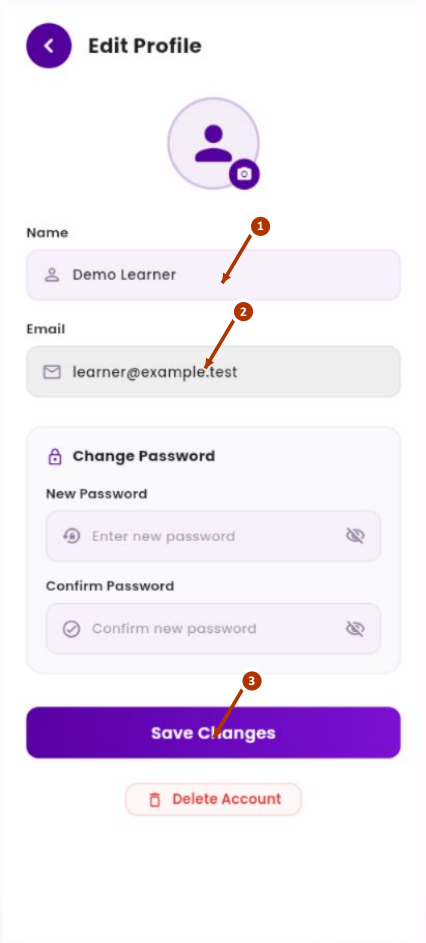
1. Correct answer

2. Explanation

3. Continue

Edit Profile

Change name and avatar here. Email is read-only. Password uses a new-password/confirmation flow; account deletion is a separate permanent action.



Actual source preview - local demo data - live values may differ

1. Avatar and name

2. Read-only email

3. Save profile

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

lib/quizs_app.dart

lib/l10n/app_strings.dart

lib/services/quiz_api_service.dart

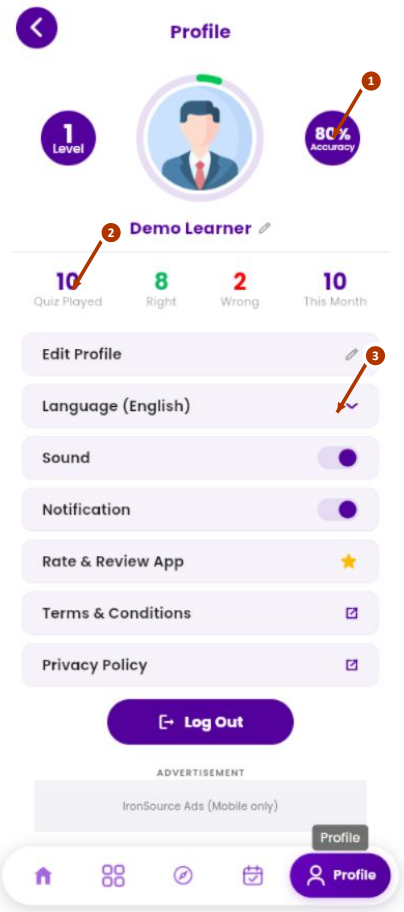
app/Services/ProfileStatsService.php

app/Services/RankingService.php

Understand every profile statistic and setting

In this version, “Quiz Played” displays questions answered today, not completed quiz sessions. Use the actual definitions below.

Screen / location: App → Profile



Actual source preview - local demo data - live values may differ

1. Level and accuracy

2. Today’s answers and monthly total

3. Language, sound, and notifications

Step by step

1. Open Profile from bottom navigation and allow statistics to load.

2. Compare the target ring, Level/Accuracy, Quiz Played, Right, Wrong, and This Month.

3. Choose English or Hindi using Language. Sound changes the saved game-sound preference.

4. Open Edit Profile to change name/avatar. Email is read-only here and must be changed through administrator user editing.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Level	user_progression.current_level, based on daily-target XP.	XP curve and earned rewards; editing total score does not directly change level.
Accuracy	Lifetime completed correct plus today's ongoing correct, divided by correct+wrong in that same scope.	Unanswered questions are excluded; dashboard and individual attempt scopes can differ.
Quiz Played	Today's completed right+wrong plus today's ongoing answer records.	Counts answered questions today; merely starting a quiz does not increase it.
Right / Wrong	Today's completed and ongoing correct/incorrect answer totals.	Driven by answer evaluation and submitted records.
This Month	Completed attempted questions plus ongoing answers in the business month, with right+wrong fallback for older tracking.	Monthly question count, not revenue or quiz-session count.
Profile ring	today_target.progress_percentage, capped at 100%.	Today's fixed target and completed quiz count.
Language	AppLanguage preference and bilingual content fetch.	Does not turn a single-language quiz into a bilingual one.
Sound	SoundService → SharedPreferences sound_enabled.	Retained across app restarts.
Notification toggle	Local profile-widget boolean.	No OS/FCM opt-out, persistence, or server save.
Rate & Review	AppReviewService and platform review flow.	Availability depends on the platform; no admin field.
Privacy / Terms	Local profile policy content and separate server public pages.	Keep both copies aligned. The old local "ad-free" text is not evidence about actual SDK integration.
Log Out	AuthService logout.	Clears local authentication/session token; protected actions require sign-in again.

Current implementation limit
Achievements rank, Home Leaderboard rank, and quiz-result rank are different contexts and need not match.

Verification after saving

Answer another question and refresh Profile to compare today's counts. Do not expect Level to increase before a target reward is earned.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Services/ProfileStatsService.php

app/Services/RankingService.php

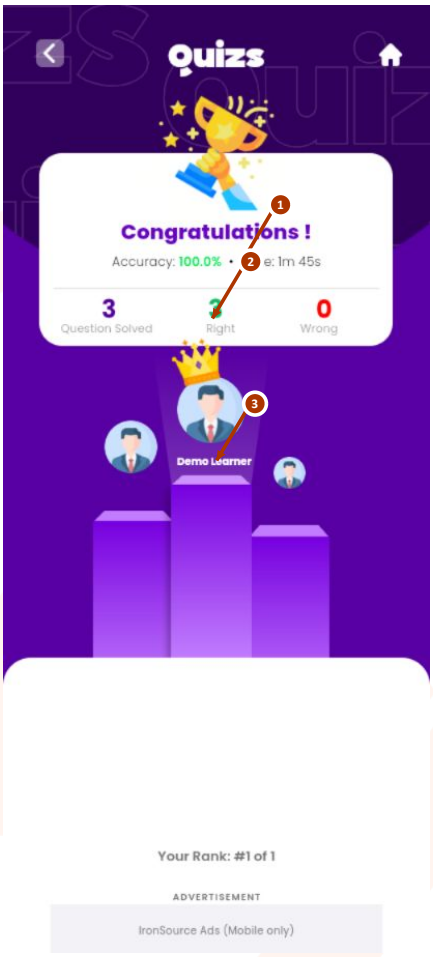
lib/screens/profile_screen.dart

lib/services/sound_service.dart

Understand results and leaderboards

Results combines quiz outcome with ranking. Performance uses correct/total questions; accuracy uses attempted answers; ranking has its own formula.

Screen / location: App → Quiz complete → Results



Actual source preview - local demo data - live values may differ

1. Accuracy and time

2. Solved / Right / Wrong

3. Ranking podium

Step by step

1. Complete the quiz and open Results.

2. Compare Solved, Right, Wrong, Accuracy, and Time with the attempt report.

3. Read Your Rank and participants. An empty podium position is not a real participant.

4. When opened from Home → Leaderboard, the screen shows aggregated completed-quiz history instead.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Accuracy	Correct ÷ attempted questions × 100.	Changes with recorded answers; skipped questions are separately unanswered.
Performance heading	Correct ÷ total_questions; defaults: excellent ≥90, good ≥70, average ≥50, otherwise low.	PerformanceMessageService and results_screen display copy; separate from passing.
Score	Sum of points for correct question answers.	Added to user total score; daily XP is a separate event.
Passing	Percentage ≥ quiz.passing_percentage.	Passing increments streak; every completed quiz counts toward daily completion.
Time	completed_at − started_at / stored duration.	Real elapsed attempt duration, not the fixed Next Trial countdown.
Quiz rank	Best completed attempt per user; score, accuracy, then elapsed time, with completion-time tie-break.	QuizRankingService avoids duplicate ranking entries for repeat players.
Home Leaderboard	Total correct across completed quizzes, with solved-count tie-break.	globalSummaryFor; separate from Achievement's level/XP hierarchy.

Verification after saving

Compare the same attempt's answers, score, and time in admin View. A network submission failure can show local fallback results; confirm server completion separately.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Api/QuizApiController.php

app/Services/QuizRankingService.php

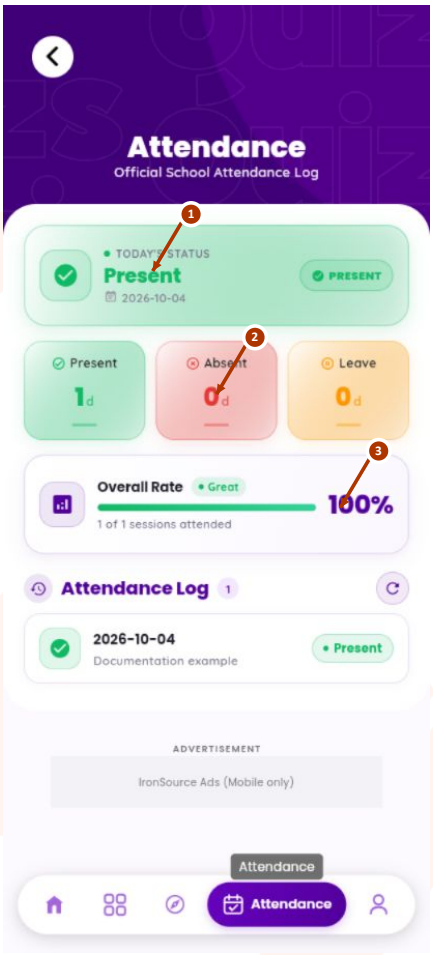
app/Services/PerformanceMessageService.php

lib/screens/results_screen.dart

Read attendance and attendance rate

Official Attendance Log comes from administrator-marked entries. Playing a quiz does not mark attendance.

Screen / location: App → Attendance



Actual source preview - local demo data - live values may differ

1. Today's status

2. Present / Absent / Leave

3. Overall rate and history

Step by step

1. Open Attendance and read Today's Status.

2. Compare Present/Absent/Leave cards with Overall Rate.

3. Inspect date, note, and status in Attendance Log; use Refresh to load history again.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Today's Status	Attendance record for the business date.	Saved for that date through Admin → Attendance.
Present / Absent / Leave	Counts of stored user statuses.	Only marked dates; unmarked calendar days do not become absent automatically.
Overall Rate	Present ÷ (present + absent + leave) × 100.	Leave is included in the denominator; zero entries show zero.
Attendance Log	Saved date, status, and note history.	Administrator notes may be visible in the app.

Verification after saving

Save a test date's status in admin and refresh the app to confirm the entry and calculated rate.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Services/AttendanceStatsService.php

lib/screens/attendance_screen.dart

Read achievements and global ranking

This ranking uses level and XP progression. Its formula differs from the single-quiz leaderboard and Home's aggregate leaderboard.

Screen / location: App → Home → Achievement



Actual source preview - local demo data - live values may differ

1. Level / accuracy

2. Ranking summary

3. Global ranking list

Step by step

1. Open the Achievement tile from Home.

2. Read Your Rank, Total Users, and active-user summaries.

3. Find your name, level, and accuracy in Global Ranking.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Level / XP	Daily-target progression records.	Configured XP is awarded on the first daily target completion.
Your Rank	Eligible users sorted by level → XP → accuracy → completed-quiz count → user ID.	Total user score is not this hierarchy's primary signal.
Total Users	Active non-admin eligible accounts.	Changes with account creation, activity, and role changes.
Active users	ActiveUsersService activity window; default 30 days.	ACTIVE_USERS_WINDOW_DAYS; not the calendar-month count.
Active This Month	Eligible users with last_active_at within the current business calendar month.	Different from the rolling activity window.
Ranking list	Top eligible users from RankingService.	Completions/progression drive ranking; names and avatars come from user records.

Verification after saving

Achievements and Profile use RankingService; compare after reload. If Home Leaderboard differs, compare the formulas first.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Api/AchievementController.php

app/Services/RankingService.php

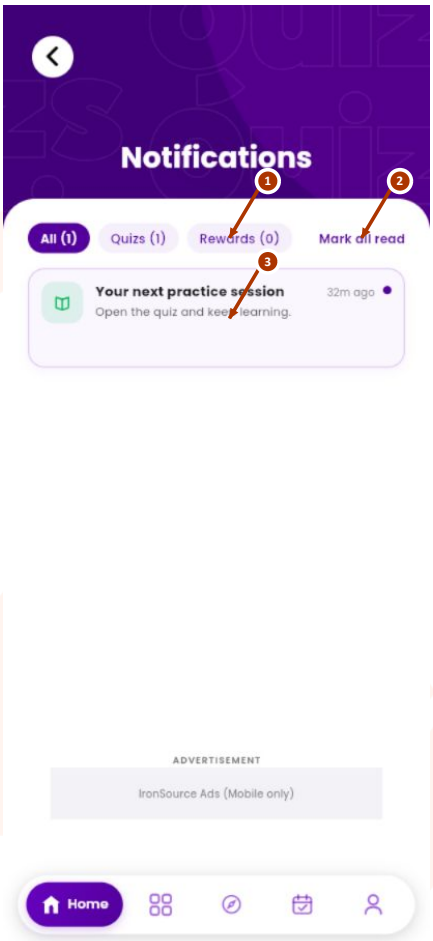
app/Services/ActiveUsersService.php

lib/screens/achievements_screen.dart

Use the inbox and read tracking

Inbox messages and recipient read states come from the server. System push delivery and inbox reading are separate events.

Screen / location: App → Bell / Profile → Notification



Actual source preview - local demo data - live values may differ

1. All / Quiz / Rewards

2. Read action

3. Message and destination

Step by step

1. Open Home’s bell or Profile’s Notification row.

2. Switch All, Quiz, and Rewards filters. Rewards includes achievement-destination messages.

3. Tap a message to mark it read and open its configured destination.

4. Use Mark All as Read for unread messages and pull to refresh for new entries.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
All / Quiz / Rewards	Client filter using destination_type.	Filtering does not delete messages.
Unread indicator	Recipient read_at state.	Changed by tap/read-all APIs; dismissing an OS banner is not reliable proof of reading.
Relative time	Client display of sent/created timestamps.	Not a measurement of phone-delivery latency.
Message / Thumbnail	Notification title, body, and image_url.	Configured in Admin → New Notification.
Tap destination	Shared notification_navigation.dart.	Quiz opens question flow; Daily Target opens Profile; invalid quiz falls back to categories.

Verification after saving

Read a message and compare its recipient Read timestamp in admin View. Check destination behavior with the intended account.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers/Api/NotificationController.php

lib/screens/notifications_screen.dart

lib/services/notification_navigation.dart

Find every customization point

Identify the source before changing any text, number, chart, or image. Use this map to choose the correct form, server configuration, or app file and understand rollout behavior.

Screen / location: Admin settings / Server config / Flutter source

Step by step

1. Find the item in the table.

2. Save admin content through its form. Change server configuration/environment and refresh configuration caches when required.

3. For Flutter code/assets, modify source, rebuild, and distribute the app update.

4. Verify a real user flow and the related report.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
Category title / image / color	Admin → Categories.	Next content fetch; no new app binary.
Quiz title / duration / difficulty	Admin → Quizzes → Edit.	Metadata on the next quiz load.
Questions / answers / translations	Admin → Questions Bank → Edit.	Next fetch; submitted scores are not automatically recalculated.
Notification content / audience / navigation	Admin → Push Notifications → New.	New inbox records and an FCM send attempt.
Daily goal	Settings → General; individual overrides in Users → Edit.	New daily rows; existing history is fixed.
Attendance	Admin → Attendance.	Selected date's status and attendance statistics.
Dashboard numbers / chart series	Computed from attempts, users, questions, and daily progress.	Change the actual underlying activity/content; chart labels are code.
OTP / email branding	SMTP settings; resources/views/emails/*.blade.php.	Server-rendered mail; app strings remain separate.
Online heartbeat / timeout	config/quiz.php; lib/services/activity_service.dart.	Activity timing and online estimate; no admin numeric control.
XP economy / levels / result thresholds	config/quiz.php; LevelService / ProgressionService / PerformanceMessageService.	Server formulas, not admin sliders.
Global / quiz ranking	RankingService.php / QuizRankingService.php.	Separate formulas; manual user score alone does not control both.
API base domain	lib/services/api_client.dart → ApiClient.baseUrl.	New mobile build; verify https://your-domain/api and public storage URLs.
Logo / splash / app name / fonts	Flutter assets, pubspec.yaml, and platform files.	New app build; backend public/logo.png is separate.
Language / interface copy	lib/l10n/app_strings.dart and screens.	New app build; question translations are admin content.
Ads	lib/services/iron_source_service.dart and native SDK settings.	New build and valid provider units; separate from offerwall records.
Privacy / Terms / Delete account	Server LegalController and resources/views/legal.	Server pages; also verify app policy copy, links, and domain.

Field / Feature	Definition and source	Effect of changes
App update behavior	lib/services/app_update_service.dart.	Android Play in-app updates; no administrator version toggle.

Verification after saving

Choose the correct source so unrelated records are not edited. Distinguish a backend save from a native binary release throughout rollout.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

config/quiz.php

lib/services/api_client.dart

lib/services/iron_source_service.dart

lib/l10n/app_strings.dart

Troubleshoot common problems

Check the intended user, server domain, and date/range first, then isolate the issue using the table.

Screen / location: Symptom → Source → Resolution

Step by step

1. Find the symptom and open its related chapter.

2. Check admin save success, API response, and app refresh in that order.

3. Inspect storage/logs/laravel.log for server errors. Keep private credentials and personal data out of public documentation.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
New category / quiz missing	Active state, parent category, language, sort order, or API domain.	Reload and confirm questions exist and content was saved on the intended server.
Broken artwork	Storage link, APP_URL, public URL, or permissions.	Run php artisan storage:link and verify the URL, then fetch again.
Hindi content appears in English	Missing Hindi translation or a single-language quiz.	Fill the Hindi tab in a bilingual quiz and account for fallback behavior.
Today's target unchanged	Existing day's fixed effective_target or custom user override.	Check a new daily row and custom/global priority.
Unexpected dashboard total	Chart range versus today/lifetime cards; created_at buckets; users versus plays.	Compare definitions against attempt and content records.
Inbox present, phone push absent	Token, permission, credentials/project, or FCM failure.	Verify real-device registration and Firebase setup; sent status is not delivery proof.
Email missing	SMTP/From settings, stopped worker, or spam.	Test Send Test, Save Changes, and queue processing separately.
Offerwall empty / unavailable	Disabled configuration or absent completion callback.	Check enabled/key/secret, signed callback, and completed_at filters.
Temporary account restricted	must_change_password is true.	Complete the user's Force Change Password flow.
Missing table / column	Release code with pending migrations.	Back up, apply migrations, and inspect the actual error.
Ads / Google / updates unavailable in browser	Native integrations are not validated by the browser build.	Test a signed native build with provider configuration.

Verification after saving

Before release, test category → quiz → questions → attempt → results → target/profile → notification → attendance.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

app/Http/Controllers

lib/services

config

Flutter Mobile Application Setup

Configuring and running the mobile app located in `Quiz Application/`.

Open the `Quiz Application` folder.

```
flutter pub get
```

Open `lib/services/api_client.dart` and set your live backend endpoint:

```
class ApiClient {
ApiClient._();
static final ApiClient instance = ApiClient._();

// Set your active production server URL:
static const String baseUrl = 'https://quizs.in/api';

// For Local Emulator testing use: 'http://10.0.2.2:8000/api'
```

```
flutter run
```

Firebase & Google Sign-In Fix (SHA-1 Setup)

How to register cryptographic SHA-1 fingerprints in Firebase Console to prevent Google Sign-In Error 10.

1. Get Debug SHA-1

```
cd "Quiz Application/android"
./gradlew signingReport
```

Copy the SHA-1 fingerprint under `Variant: debug`.

2. Get Google Play Production SHA-1

In Google Play Console, go to **Release -> Setup -> App Integrity -> App signing key certificate** and copy the SHA-1.

3. Add to Firebase

Go to **Firebase Console -> Project Settings -> Your Android Apps -> Add Fingerprint** and paste both keys. Download the updated `google-services.json` and place it in `Quiz Application/android/app/google-services.json`.

Configure banner and interstitial ads

This source integrates Unity LevelPlay banner and interstitial ads. Payouts and revenue are provider-account matters, not fixed Quizs application features.

Screen / location: Flutter → IronSourceService / banner slot

Step by step

1. Set your appKey, interstitialAdUnitId, and bannerAdUnitId in lib/services/iron_source_service.dart.

2. Match native platform integration and mediation adapters to the provider configuration.

3. Test load/show in a signed native build. The browser preview uses a Mobile only placeholder.

4. Configure offerwall launch and callbacks separately using the Offerwall chapter.

Field definitions and change impact

Field / Feature	Definition and source	Effect of changes
appKey	LevelPlayInitRequest in IronSourceService.	Changes SDK initialization; not an administrator Settings field.
bannerAdUnitId	Banner slot and LevelPlay banner API.	Requires a new native build and valid provider placement.
interstitialAdUnitId	Interstitial load/show flow.	Provider unit and call sites determine presentation.
Revenue / Payout	Provider account and contract.	Quizs admin has no fixed payout calendar or withdrawal tool.

Verification after saving

Check real-device load/show and failure states with provider test configuration. A documentation placeholder is not a paid impression.

Technical sources

Server paths are inside Quiz Application Admin; lib paths are inside Quiz Application.

lib/services/iron_source_service.dart

lib/widgets/iron_source_banner_slot.dart

pubspec.yaml

Compiling APK & Play Store (AAB)

How to build production-ready APK binaries for direct device testing and Google Play Store App Bundles (AAB).

1. Direct Release APK

```
cd "Quiz Application"
flutter build apk --release
```

Binary generated at: `Quiz Application/build/app/outputs/flutter-apk/app-release.apk`

2. Google Play Store App Bundle (AAB)

```
cd "Quiz Application"
flutter build appbundle
```

Bundle generated at: `Quiz Application/build/app/outputs/bundle/release/app-release.aab`

Software Delivery Agreement & Copyright Notice

Official Software Delivery Agreement, Intellectual Property Attribution, and Commercial License Terms.

Document No: SDA-ARKNOX-2026-09001

Date: 02 September 2026 | **Quotation Ref:** ARKNOX/QT/2026-09/001

Developer/Licensor: Arknox | **Client/Licensee:** Krishna (krishnar8111@gmail.com)

1. Parties to the Agreement

Developer / Licensor:
Arknox
UDYAM: UDYAM-TR-02-0007126 | ISO 9001:2015: E20240913795 | Trademark: 11157137 | GST: 16BIHPH4252H1ZE
Registered Office: KHATIYAN NO-2785, BLOCK TEPANIYA RD BLOCK, NEAR PUMP HOUSE, DHAJANAGAR GUAN, KHAMAR BARI, Udaipur, Gomati, Tripura - 799114
Email: info@arknox.in | Phone: +91-8787388949

Client / Licensee:
Krishna (Email: krishnar8111@gmail.com | Phone: +91 77888 72183)
Product: Quizs - Smart work smarter you (Mobile App & Admin Backend)

2. Grant of License & Ownership Transfer

Commercial Usage Rights: The Client is granted perpetual, unrestricted, worldwide commercial rights to deploy, operate, customize, and monetize the Quizs application across public app stores (Google Play Store and Apple App Store) and private hosting servers.

Source Code Ownership: The full delivered codebase (Flutter Mobile Application, Laravel 12 Backend & Web Admin Panel, and database schemas) is transferred to the Client for business operations, maintenance, and future feature expansions.

Restrictions: Raw redistribution or marketplace resale of the uncompiled source code template under Arknox's name is prohibited without express written consent.

15-Day Warranty & Support: Arknox provides 15 days of complimentary bug-fixing support starting from the formal handover date covering agreed deliverables.

3. Developer Contact Information

Company: Arknox

Official Website: www.arknox.in

Support Email: info@arknox.in

Helpline: +91-8787388949

Business Hours: Monday to Friday, 9:00 AM to 6:00 PM IST

Conclusion Section

Use this guide to configure Quizs, verify the actual app behavior, and keep administrator content, server configuration, and native app changes in their correct places.

Administrator access

Open your own backend domain followed by /admin/login. Use the credentials created for that deployment. Accounts, passwords, and URLs from unrelated reference projects do not apply to Quizs.

Release verification

Complete a category, quiz, question, attempt, result, target/profile, notification, and attendance test. Confirm native integrations separately on the intended device and provider account.

Contact for support

Email: info@arknox.in
Support Center: +91-8787388949

Arknox